



University of
Applied Sciences
St. Pölten

Orchestrierung von Large Language Models im Kontext von Energiegemeinschaften

Evaluation deterministischer, planbasierter und iterativer Tool-
Orchestrierung

Bachelorarbeit

Angestrebter akademischer Grad

Bachelor of Science (BSc)

eingereicht von

David Schwarz

52308568

im Rahmen des
Studienganges Data Science and Business Analytics an der Hochschule für Angewandten
Wissenschaften St. Pölten

Betreuung

Betreuer/in: FH-Prof. Dipl.-Wirt.-Inf. Dr. Torsten Priebe

Kurzfassung

Enthält:

- Motivation,
- Problemstellung,
- Ergebnisse
- (Inhalt siehe englische Version)

Inhaltsverzeichnis

1. Einleitung	7
1.1. Problemfeld	7
1.2. Forschungsproblem	7
1.3. Forschungsfrage	8
1.4. Ziel der Arbeit	8
1.5. Hypothese	8
1.6. Methodisches Vorgehen	8
2. Grundlagen	11
2.1. Large Language Models	11
2.2. Tool Calling	11
2.3. Agenten / Multi-Agent-Systeme	11
2.4. 2.1 Large Language Models	11
2.5. 2.2 Tool Calling	12
2.6. 2.3 Agenten / Multi-Agent-Systeme	12
3. Stand der Forschung	13
3.1. Datenzugriff	13
3.2. Zeitreihendaten	13
3.3. Toolnutzung	14
3.4. Orchestrierung	16
3.5. Evaluation	20
3.6. Zwischenfazit	23
4. Methodik	25
4.1. Forschungsdesign	25
4.2. Literaturrecherche	25
4.3. Technische Rahmenbedingungen	26
4.4. Datengrundlage	27
4.5. Tooldesign	30
4.6. Orchestrierungsstrategien	33
4.7. Modellauswahl	37
4.8. Aufgabenauswahl	39
4.9. Bewertungskriterien	40
4.10. Versuchsablauf	45
5. Ergebnisse	47
5.1. Gesamtvergleich der Methoden	47
5.2. Analyse der Korrektheits- und Effizienzkomponenten	48
5.3. Gesamtvergleich der Aufgabentypen	48
5.4. Vergleich der Modelle	49
5.5. Vergleich der Modelle und Methoden	49

5.6. Fehlerrate	50
5.7. Laufzeit	51
5.8. Tokenverbrauch	51
6. Diskussion	53
6.1. Brainstorming	53
6.2. Warum die Tokenverbräuche unterschiedlich sind	54
6.3. Bedeutung der hohen Fehlerrate bei der iterativen Methode	54
6.4. Zusammenhang zwischen Iterationen und Timeout	54
7. Fazit	57
7.1. Brainstorming	57
7.2. Weiterführende Arbeiten	57
7.3. Brainstorming	57
Akronyme	59
Abbildungsverzeichnis	61
Tabellenverzeichnis	63
Source Code Verzeichnis	65
Literaturverzeichnis	67

1. Einleitung

1.1. Problemfeld

Large Language Models (LLMs) werden zunehmend eingesetzt, um komplexe Informationsanfragen zu verarbeiten und Aufgaben auf Basis natürlicher Sprache zu bearbeiten [1], [2]. Durch die Integration externer Werkzeuge können Sprachmodelle beispielsweise auf strukturierte Daten zugreifen, Berechnungen durchführen oder weitere Verarbeitungsschritte auslösen [2], [3], [4]. Die dabei eingesetzte Orchestrierung bestimmt, welche Werkzeuge für eine Anfrage verwendet werden und wie deren Aufrufe miteinander koordiniert werden [5], [6].

Energiegemeinschaften sind Zusammenschlüsse von Teilnehmenden, zu denen beispielsweise Privatpersonen, Gemeinden oder Unternehmen gehören können. Sie ermöglichen die gemeinsame Produktion, Speicherung und Nutzung von Energie mit dem Ziel, lokal erzeugte Energie möglichst effizient zu nutzen [7], [8]. Für die Verwaltung und Analyse einer Energiegemeinschaft fallen dabei unter anderem Verbrauchs- und Erzeugungsdaten verschiedener Zählpunkte an. Auf Grundlage dieser Daten können beispielsweise Fragen zu aktuellen oder vergangenen Verbrauchs- und Erzeugungsmengen, Kennzahlen oder kombinierten Energieflüssen gestellt werden.

Eine mögliche Schnittstelle zur Nutzung dieser Daten stellt ein Chatbot dar, über den Teilnehmende ihre Anfragen in natürlicher Sprache formulieren können. Die Herausforderung besteht dabei nicht ausschließlich darin, die Anfrage inhaltlich zu verstehen. Das System muss zusätzlich bestimmen, welche Daten und Werkzeuge für die Bearbeitung benötigt werden und wie deren Ergebnisse zu einem korrekten Ergebnis zusammengeführt werden.

Dabei kann die Orchestrierung unterschiedlich stark autonomisiert werden. Bei einer deterministischen Orchestrierung werden die möglichen Verarbeitungsschritte weitgehend durch vorgegebene Regeln bestimmt. Autonomere Ansätze übertragen dagegen einen größeren Teil der Entscheidungsfindung auf das LLM, das beispielsweise selbst geeignete Werkzeuge auswählen, Verarbeitungsschritte planen oder auf Zwischenergebnisse reagieren kann. Dadurch können flexiblere Abläufe ermöglicht werden, gleichzeitig steigt jedoch der Entscheidungsspielraum des Modells und damit die Möglichkeit unerwarteter oder fehlerhafter Toolaufrufe [9], [4], [6], [10].

1.2. Forschungsproblem

Mit zunehmendem Autonomiegrad kann ein LLM einen größeren Teil der Tool-Orchestrierung selbst übernehmen. Dies ermöglicht eine flexible Reaktion auf unterschiedliche und komplexe Anfragen, kann jedoch auch zu unnötigen oder fehlerhaften Toolaufrufen führen. Beispielsweise könnte ein Modell bereits vorhandene Informationen erneut abrufen oder Verarbeitungsschritte durchführen, die für die Beantwortung einer Anfrage nicht erforderlich sind. Dadurch können zusätzlicher Ressourcenbedarf und längere LLM-Laufzeiten entstehen.

Darüber hinaus können fehlerhafte Entscheidungen bei der Toolauswahl oder bei der Verarbeitung der Ergebnisse zu falschen Antworten führen. Insbesondere bei Aufgaben, für deren Bearbeitung mehrere Datenquellen miteinander verknüpft werden müssen, steigt die Anzahl der Entscheidungen, die während der Orchestrierung getroffen werden müssen. Gleichzeitig kann eine stark regelbasierte Orchestrierung zwar die möglichen Abläufe stärker kontrollieren, bei komplexeren oder nicht vollständig vorhersehbaren Anfragen jedoch weniger flexibel sein.

Für den Einsatz in Energiegemeinschaften ist eine zuverlässige Verarbeitung der Energiedaten von besonderer Bedeutung. Die aus den Daten abgeleiteten Informationen können beispielsweise als Grundlage für Entscheidungen der Teilnehmenden dienen. Fehlerhafte oder unvollständige Ergebnisse können dadurch zu falschen Einschätzungen der eigenen Energieerzeugung oder des Verbrauchs führen. Die Wahl einer geeigneten Orchestrierungsstrategie stellt somit einen Zielkonflikt zwischen der Flexibilität autonomer Verfahren und der Kontrolle durch vorgegebene Abläufe dar.

1.3. Forschungsfrage

Wie beeinflusst der Grad der Autonomie bei der Tool-Orchestrierung von Large Language Models die Genauigkeit und den Ressourcenbedarf bei der Analyse von Zeitreihendaten in Energiegemeinschaften?

1.4. Ziel der Arbeit

Ziel dieser Arbeit ist es, den Einfluss unterschiedlicher Autonomiegrade bei der Tool-Orchestrierung von Large Language Models auf die Bearbeitung datenbasierter Aufgaben in Energiegemeinschaften systematisch zu untersuchen. Hierzu werden deterministische, planbasierte und iterative Orchestrierungsansätze hinsichtlich ihrer Genauigkeit und ihres LLM-seitigen Ressourcenbedarfs verglichen. Dabei soll untersucht werden, ob ein höherer Autonomiegrad bei der Bearbeitung komplexerer Aufgaben einen Vorteil bietet und wie sich dieser auf die benötigten LLM-Ressourcen auswirkt.

1.5. Hypothese

Es wird angenommen, dass ein höherer Autonomiegrad insbesondere bei Aufgaben mit mehreren voneinander abhängigen Verarbeitungsschritten Vorteile bei der flexiblen Bearbeitung bietet. Gleichzeitig wird erwartet, dass durch die zusätzliche Entscheidungsfreiheit des LLM der Ressourcenbedarf und die Wahrscheinlichkeit fehlerhafter Toolentscheidungen steigen können.

1.6. Methodisches Vorgehen

Diese Arbeit verfolgt einen experimentellen und vergleichenden Forschungsansatz. Auf Basis einer Literaturrecherche werden zunächst bestehende Ansätze zur Orchestrierung von LLMs und zur Tool-Nutzung untersucht. Darauf aufbauend werden drei Orchestrierungsansätze mit unterschiedlichen Autonomiegraden implementiert und für die Untersuchung verwendet.

Für die experimentelle Evaluation werden drei Prototypen entwickelt, deren Datenstruktur sich an einer realen Energiegemeinschaft orientiert. Die für die Versuche verwendeten Energiedaten werden synthetisch erzeugt und bilden ausgewählte Anforderungen einer realen Energiegemeinschaft ab. Die Evaluation umfasst 90 Aufgaben, die in direkte Datenabfragen, Einzelquellenverarbeitung und Mehrquellenverarbeitung unterteilt sind. Die Aufgaben werden mit einem kleineren lokalen Modell, einem größeren lokalen Modell und einem über eine API bereitgestellten Cloud-Modell bearbeitet. Dabei

werden für alle Modelle und Orchestrierungsansätze dieselben Aufgaben, Prompts und Werkzeuge verwendet.

Die Ergebnisse der einzelnen Durchläufe werden anschließend hinsichtlich der zuvor definierten Bewertungskriterien ausgewertet. Ergänzend werden der Tokenverbrauch und die Laufzeit der LLM-Aufrufe erfasst, um den Ressourcenbedarf der verschiedenen Orchestrierungsansätze zu vergleichen. Auf diese Weise soll der Einfluss des Autonomiegrades unter kontrollierten Versuchsbedingungen untersucht und die Eignung der unterschiedlichen Ansätze für die Bearbeitung von Tool-basierten Aufgaben im Kontext von Energiegemeinschaften bewertet werden.

2. Grundlagen

2.1. Large Language Models

Inferenz

Inferenz bedeutet im Zusammenhang mit KI, dass ein bereits trainiertes Modell der künstlichen Intelligenz neue, unbekannte Daten verarbeitet. Es nutzt das während der Lernphase angeeignete Wissen, um eigenständig Vorhersagen, Entscheidungen oder Antworten zu erzeugen [11].

2.2. Tool Calling

Large Language Models können durch die Anbindung externer Werkzeuge auf Informationen und Funktionen zugreifen, die nicht allein durch das im Modell enthaltene Wissen bereitgestellt werden können. Werkzeuge können dabei beispielsweise den Zugriff auf externe Datenquellen, Softwarefunktionen oder andere digitale Systeme ermöglichen. Damit ein Sprachmodell ein Werkzeug verwenden kann, wird dieses über eine strukturierte Beschreibung bereitgestellt. Eine solche Beschreibung umfasst beispielsweise einen eindeutigen Namen, eine Beschreibung der Funktion, Einschränkungen sowie die erwartete Struktur der Eingabeparameter und des Rückgabeformats. Auf Grundlage dieser Beschreibung kann das Modell ein geeignetes Werkzeug auswählen und ein entsprechendes Argumentobjekt für dessen Aufruf erzeugen [12].

Der erzeugte Tool-Aufruf wird anschließend von einer ausführenden Umgebung verarbeitet. Die Ausführung eines Werkzeugs erzeugt dabei ein Ergebnis beziehungsweise eine Rückmeldung, beispielsweise in Form von Text, strukturierten Daten oder einem Fehlercode. Dieses Ergebnis kann dem Sprachmodell anschließend wieder als Beobachtung zur Verfügung gestellt werden und damit als Grundlage für die weitere Verarbeitung dienen [12].

2.3. Agenten / Multi-Agent-Systeme

- falls für deine Arbeit erforderlich

2.4. 2.1 Large Language Models

- LLMs erzeugen ihre Ausgaben tokenweise und sind daher nicht mit deterministischen Programmlogiken gleichzusetzen.
- Ein LLM kann strukturierte Ausgaben erzeugen, beispielsweise JSON-basierte Tool Calls, benötigt dafür aber eine entsprechende Promptstruktur.
- Die Fähigkeit, ein Tool korrekt auszuwählen, bedeutet nicht automatisch, dass das Modell auch einen mehrstufigen Prozess zuverlässig steuern kann.

- Größere Modelle scheinen in den Experimenten besser mit komplexeren, mehrstufigen Entscheidungsprozessen umgehen zu können.
- Besonders relevant ist die Fähigkeit des Modells, Informationen aus vorherigen Verarbeitungsschritten in nachfolgenden Entscheidungen zu berücksichtigen.
- Bei lokalen Modellen zeigen sich deutliche Unterschiede zwischen den Modellgrößen.
- Qwen3:8B kann einfache Aufgaben bzw. einzelne Planungsschritte grundsätzlich bewältigen.
- Qwen3:30B zeigt beim iterativen Vorgehen ein deutlich robusteres Verständnis des grundsätzlichen Ablaufs.
- Modellgröße kann nicht nur die Qualität der sprachlichen Antwort beeinflussen, sondern auch die Fähigkeit zur Steuerung eines Tool-gestützten Workflows.

2.5. 2.2 Tool Calling

- Die Tools werden mit ihren Funktionen und Argumenten beschrieben.
- Die Tool-Beschreibungen sind für das Modell grundsätzlich ausreichend, um zumindest im einfachen Planungsfall passende Tools auszuwählen.
- Bei der plan-basierten Methode konnte das Modell aus Aufgabe und Tooldefinitionen einen plausiblen Plan erzeugen.
- Damit scheint die reine Tool Selection bei einfachen Aufgaben nicht das zentrale Problem zu sein.
- Die Schwierigkeiten treten stärker auf, sobald bereits ausgeführte Tools und deren Ergebnisse in den Entscheidungsprozess einbezogen werden.
- Das erfolgreiche Erzeugen eines einzelnen Tool Calls bedeutet nicht automatisch, dass ein Modell einen mehrstufigen Tool-Calling-Prozess zuverlässig steuern kann.

2.6. 2.3 Agenten / Multi-Agent-Systeme

- Beim plan-basierten Ansatz erstellt das LLM zunächst einen vollständigen Plan, der anschließend durch das System abgearbeitet werden kann.
- Beim iterativen Ansatz entscheidet das LLM schrittweise und erhält nach jeder Tool-Ausführung neue Informationen.
- Der iterative Ansatz stellt damit höhere Anforderungen an das LLM.
- Das Modell muss den bisherigen Zustand erfassen.
- Das Modell muss vorhandene Ergebnisse berücksichtigen.
- Das Modell muss fehlende Informationen erkennen.
- Das Modell muss weitere Tools auswählen.
- Das Modell muss bereits erledigte Schritte erkennen und nicht unnötig wiederholen.
- Das Modell muss erkennen, wann die Aufgabe abgeschlossen ist.

3. Stand der Forschung

3.1. Datenzugriff

3.1.1. Text to SQL

Eine Möglichkeit, Datenbanken über natürliche Sprache zugänglich zu machen, ist Text-to-SQL. Dabei wird eine natürlichsprachliche Anfrage in eine ausführbare SQL-Abfrage übersetzt [13].

Die Generierung einer geeigneten SQL-Abfrage erfordert neben der Interpretation der natürlichsprachlichen Anfrage auch ein Verständnis des zugrunde liegenden Datenbankschemas. Moderne Text-to-SQL-Systeme beziehen daher Schemainformationen ein und verwenden unter anderem Verfahren zum Schema Linking, um relevante Tabellen und weitere Datenbankelemente für die Abfragegenerierung zu bestimmen [13].

Mit zunehmender Komplexität der Datenbank entstehen dabei zusätzliche Herausforderungen. Komplexe Schemata, Beziehungen zwischen Tabellen, mehrdeutige Anfragen sowie die korrekte Generierung komplexer SQL-Konstruktionen können die Zuverlässigkeit der erzeugten Abfragen beeinträchtigen. Entsprechend sind Schemaverständnis, robuste SQL-Generierung und die Behandlung komplexer Anfragen weiterhin zentrale Herausforderungen der Text-to-SQL-Forschung [14].

3.1.2. Function Calling

Eine alternative Möglichkeit besteht darin, Datenbankoperationen über vordefinierte Funktionen bereitzustellen. Dabei wird die für eine Operation benötigte Abfragelogik innerhalb einer Funktion gekapselt, während das Sprachmodell lediglich aus den verfügbaren Funktionen auswählt und deren definierte Parameter bestimmt. De Costa et al. verwenden einen solchen Ansatz, bei dem zweckgebundene Funktionen vorab definierte und validierte SQL-Abfragen kapseln. Dadurch wird die Generierung der eigentlichen SQL-Logik aus dem direkten Einflussbereich des Sprachmodells genommen und die zulässige Interaktion mit der Datenbank auf die bereitgestellten Operationen beschränkt [15].

Der Ansatz bietet insbesondere dort Vorteile, wo die Kontrollierbarkeit und Nachvollziehbarkeit der Datenabfragen relevant sind. De Costa et al. berichten in ihrem konkreten Anwendungskontext zudem eine höhere durch Experten bewertete Korrektheit sowie Vorteile hinsichtlich der Wartbarkeit gegenüber einem direkten NL-to-SQL-Ansatz. Gleichzeitig entsteht durch die Entwicklung und Pflege der Funktionsbibliothek ein zusätzlicher Aufwand [15].

3.2. Zeitreihendaten

Energiedaten liegen häufig in Form von Zeitreihen vor. Eine direkte Übergabe längerer Zeitreihen an ein LLM kann problematisch sein. Bei der numerischen Repräsentation entstehen einerseits hohe Tokenkosten, andererseits können relevante zeitliche, dimensionale oder frequenzbezogene Merkmale

nur schwer aus der Folge numerischer Werte extrahiert werden. Liu et al. zeigen beispielsweise, dass ein einzelnes Zeitreihenbeispiel im untersuchten RCW-Datensatz bei GPT-4o bis zu 60.000 Input-Tokens benötigen kann. Die Autoren identifizieren die direkte numerische Modellierung daher als eine wesentliche Ursache für die beobachteten Schwierigkeiten von LLMs beim Reasoning über Zeitreihen [16].

Eine Möglichkeit besteht darin, Zeitreihen zunächst durch spezialisierte Verfahren zu analysieren und dem LLM lediglich relevante Merkmale oder Ergebnisse bereitzustellen. Dies kann beispielsweise die Berechnung statistischer Kennwerte umfassen. Für komplexere Fragestellungen sind jedoch spezifische Verfahren zur Merkmalsextraktion oder Mustererkennung erforderlich, wodurch die Lösung stärker auf die jeweils vorgesehenen Fragestellungen zugeschnitten werden muss [16].

Einen alternativen Ansatz untersuchen Liu et al. mit VL-Time. Anstatt die Zeitreihen direkt als numerische Daten an das LLM zu übergeben, werden diese visualisiert und anschließend von einem multimodalen LLM verarbeitet. Die Methode umfasst zunächst eine Planungsphase, in der unter anderem eine geeignete Visualisierung im Zeit- oder Frequenzbereich sowie relevante Merkmale bestimmt werden. In der anschließenden Lösungsphase wird die visualisierte Zeitreihe zusammen mit sprachlichen Hinweisen verarbeitet [16].

Die Visualisierung dient dabei nicht nur der Darstellung, sondern gleichzeitig als Kompromiss zwischen Informationsgehalt und Kontextgröße. Im untersuchten RCW-Beispiel reduziert sich die Repräsentation eines Zeitreihenbeispiels von bis zu 60.000 Tokens bei numerischer Modellierung auf 85 Tokens bei visueller Modellierung. Zusätzlich untersuchen Liu et al. Few-Shot In-Context Learning, bei dem Beispiele zur jeweiligen Aufgabe in den Kontext aufgenommen werden. Im Vergleich zur numerischen Modellierung erzielt VL-Time dabei eine durchschnittliche relative Leistungssteigerung von 140 % und reduziert die durchschnittlichen Tokenkosten um 99 % [16].

3.3. Toolnutzung

Large Language Models können durch die Anbindung externer Werkzeuge auf Informationen und Funktionen zugreifen, die nicht allein durch das im Modell gespeicherte Wissen bereitgestellt werden können. Frühe Arbeiten untersuchten dabei zunächst die grundlegende Fähigkeit, geeignete Werkzeuge auszuwählen und korrekte API-Aufrufe zu erzeugen. Xu et al. ordnen unter anderem Toolformer und ReAct dieser frühen Entwicklung zu, in der insbesondere die Auswahl eines Werkzeugs und die korrekte Formatierung eines API-Aufrufs im Vordergrund stehen [12].

3.3.1. Tool Calling und Toolauswahl

Ein früher Ansatz zur systematischen Nutzung externer Werkzeuge durch Sprachmodelle ist Toolformer. Dabei wird untersucht, wie ein Sprachmodell selbstständig lernen kann, externe Werkzeuge über APIs einzusetzen. Das Modell soll dabei nicht nur geeignete Werkzeuge auswählen, sondern auch entscheiden, wann ein API-Aufruf sinnvoll ist und welche Argumente dafür erzeugt werden müssen. Dazu werden zunächst potenzielle API-Aufrufe in Texte eingefügt, ausgeführt und anschließend anhand ihres Beitrags zur Vorhersage zukünftiger Tokens gefiltert. Die verbleibenden Beispiele werden für die Feinabstimmung des Modells verwendet. Toolformer zeigt damit, dass Toolnutzung nicht zwingend durch eine explizite Vorgabe des verwendeten Werkzeugs erfolgen muss. Das Modell kann vielmehr lernen, selbst zu entscheiden, ob und wann externe Informationen benötigt werden. In den Experimenten führte dies unter anderem bei wissensintensiven Aufgaben zu deutlichen Verbesserungen gegenüber vergleichbaren Modellen ohne Toolnutzung [17].

Die Frage der Toolauswahl wird auch unabhängig von der konkreten Ausführung eines Tool Calls untersucht. Huang et al. führen mit MetaTool einen Benchmark ein, der bewertet, ob ein Sprachmodell erkennt, wann ein externes Werkzeug benötigt wird und welches Werkzeug für eine Anfrage geeignet ist. Dabei werden sowohl einzelne Werkzeuge als auch Szenarien mit mehreren Werkzeugen betrachtet. Die Ergebnisse zeigen, dass selbst leistungsfähige Sprachmodelle weiterhin Schwierigkeiten bei der zuverlässigen Toolauswahl aufweisen [5].

Damit verschiebt sich der Fokus von der reinen Fähigkeit, einen einzelnen API-Aufruf korrekt zu erzeugen, zunehmend auf die Frage, welches Werkzeug für eine konkrete Aufgabe geeignet ist. Das Xu-Survey beschreibt diese Entwicklung als Übergang von einer einfachen Toolauswahl hin zu komplexeren Entscheidungen über mehrere voneinander abhängige Werkzeuge [12].

3.3.2. Tool Learning und Training

Neben Ansätzen, die Toolnutzung während der Ausführung ermöglichen, wurde untersucht, Toolnutzung gezielt in den Parametern eines Sprachmodells zu verankern. Ein Beispiel hierfür ist GPT4Tools. Der Ansatz verfolgt das Ziel, kleinere Open-Source-Modelle mithilfe von Trainingsdaten eines leistungsfähigeren Sprachmodells zur Nutzung von Werkzeugen zu befähigen. Hierzu erzeugt GPT-3.5 anhand von Toolbeschreibungen und visuellen Kontexten zunächst ein instruktionales Trainingsdatenset. Die daraus erzeugten Beispiele werden anschließend verwendet, um Modelle wie OPT-13B, LLaMA-13B und Vicuna-13B mittels LoRA anzupassen [2].

Die erzeugten Trainingsbeispiele enthalten unter anderem die Entscheidung, ob ein Werkzeug benötigt wird, den Namen des aufzurufenden Werkzeugs und die entsprechenden Argumente. GPT4Tools evaluiert diese Fähigkeiten getrennt über die Successful Rate of Thought (SRt), die Successful Rate of Action (SRact) und die Successful Rate of Arguments (SRargs). Die zusammengefasste Successful Rate berücksichtigt dabei nur Aufrufe, bei denen alle drei Komponenten korrekt sind. Die Ergebnisse zeigen deutliche Verbesserungen durch das Fine-Tuning. Beispielsweise steigt die Successful Rate von OPT-13B von 0 % auf 93,2 % und die von Vicuna-13B von 12,4 % auf 94,1 %. Auch bei Werkzeugen, die nicht im Training enthalten waren, erreichen die angepassten Modelle deutlich bessere Ergebnisse. Vicuna-13B erzielt auf diesen unbekannten Werkzeugen beispielsweise eine Successful Rate von 90,6 % [2].

GPT4Tools zeigt damit, dass Toolnutzung nicht ausschließlich durch Laufzeit-Prompting oder eine externe Steuerung realisiert werden muss, sondern auch als erlernte Fähigkeit eines Sprachmodells betrachtet werden kann. Gleichzeitig unterscheidet sich dieser Ansatz von Toolformer dadurch, dass GPT4Tools gezielt kleinere Open-Source-Modelle mittels selbstgenerierter Instruktionsdaten anpasst und insbesondere auch multimodale Werkzeuge berücksichtigt [2]. Das Xu-Survey ordnet GPT4Tools entsprechend als frühen Ansatz des Supervised Fine-Tunings für Toolnutzung ein. Während frühe Fine-Tuning-Verfahren insbesondere die syntaktische Korrektheit von Toolaufrufen sowie die Zuordnung von Anfragen zu APIs verbessern, verschiebt sich der Fokus neuerer Verfahren zunehmend auf große Toolmengen und die Planung von Abhängigkeiten zwischen Werkzeugen [12].

3.3.3. Mehrstufige Toolnutzung

Mit der Nutzung mehrerer Werkzeuge entsteht eine zusätzliche Herausforderung: Ein Werkzeugaufruf kann von den Ergebnissen eines vorherigen Aufrufs abhängen. Damit reicht es nicht mehr aus, einzelne Tool Calls unabhängig voneinander korrekt zu erzeugen. Stattdessen müssen Informationen aus vorherigen Aufrufen aufgenommen und für nachfolgende Aktionen verwendet werden.

Diese Grenze zeigt sich bereits bei Toolformer. Das Verfahren beschränkt sich bei der Erzeugung und Nutzung von API-Aufrufen auf einzelne Aufrufe pro Beispiel. Dadurch kann es keine verketteten Toolaufrufe erlernen, bei denen beispielsweise die Ausgabe eines Werkzeugs als Eingabe für ein weiteres Werkzeug dient. Auch eine interaktive Nutzung eines Werkzeugs, bei der beispielsweise eine Suchanfrage auf Grundlage zuvor erhaltener Ergebnisse angepasst wird, wird vom ursprünglichen Ansatz nicht unterstützt [17].

ART greift diese Herausforderung auf und erweitert die Toolnutzung um eine explizite mehrstufige Struktur. Das Framework zerlegt Aufgaben in mehrere aufeinanderfolgende Schritte und kann die Generierung an einem Tool Call unterbrechen, dessen Ausgabe in den weiteren Ablauf integrieren und anschließend die Generierung fortsetzen. Dadurch können beispielsweise zunächst Informationen über eine Suche beschafft und anschließend mit einem Codewerkzeug verarbeitet werden [1].

Damit wird eine Grenze zwischen der Nutzung einzelner Werkzeuge und der Koordination mehrerer Werkzeuge sichtbar. Während bei einzelnen Aufrufen vor allem Toolauswahl und korrekte Parametrisierung relevant sind, entstehen bei mehreren voneinander abhängigen Aufrufen zusätzliche Anforderungen wie die Modellierung von Abhängigkeiten, die Bestimmung der Ausführungsreihenfolge, Parallelisierung, Fehlerbehandlung und erneute Planung [12].

Damit liegt der Schwerpunkt der beschriebenen Ansätze zunächst auf der Fähigkeit des Sprachmodells, Werkzeuge zu verwenden: Es muss erkennen, wann ein Werkzeug benötigt wird, ein geeignetes Werkzeug auswählen und einen korrekten Aufruf mit passenden Argumenten erzeugen können. Bei mehreren aufeinander aufbauenden Werkzeugaufrufen reicht diese Fähigkeit jedoch nicht mehr aus. Es muss zusätzlich entschieden werden, welche Aktionen in welcher Reihenfolge ausgeführt werden, wie mit den Ergebnissen vorheriger Aktionen umgegangen wird und wann die Ausführung beendet oder angepasst werden sollte. Diese übergeordnete Steuerung wird im Folgenden als Tool-Orchestrierung betrachtet [12].

3.4. Orchestrierung

Bei der Nutzung mehrerer Werkzeuge reicht es nicht aus, für jeden einzelnen Schritt ein geeignetes Werkzeug auszuwählen. Die einzelnen Werkzeugaufrufe können voneinander abhängig sein, sodass sowohl ihre Auswahl als auch ihre Ausführungsreihenfolge berücksichtigt werden müssen. Lu et al. modellieren die Ausführung komplexer Tool-Nutzung daher als gerichteten azyklischen Graphen (DAG), in dem Werkzeugaufrufe als Knoten und Abhängigkeiten zwischen ihnen als Kanten dargestellt werden. Eine solche Abhängigkeit entsteht beispielsweise, wenn ein Ausgabefeld eines Werkzeugs als Eingabe für ein nachfolgendes Werkzeug dient [18].

Die Koordination mehrerer Werkzeugaufrufe kann damit als Orchestrierungsproblem betrachtet werden. Zhe et al. beschreiben Tool-Orchestrierung entsprechend nicht ausschließlich als schrittweise Auswahl einzelner Werkzeuge, sondern als ein Problem der strukturierten Ausführungssteuerung. Dabei müssen insbesondere Abhängigkeiten zwischen Werkzeugen berücksichtigt und die Ausführung so organisiert werden, dass Fehler einzelner Aufrufe nicht zwangsläufig die gesamte Ausführung beeinträchtigen [19].

3.4.1. Mehrstufige Tool-Ausführung

Eine Form der Orchestrierung besteht darin, eine Aufgabe in mehrere aufeinanderfolgende Schritte zu zerlegen und an geeigneten Stellen Werkzeuge einzusetzen. Paranjape et al. beschreiben mit Automatic Reasoning and Tool-use (ART) ein Framework, in dem solche mehrstufigen Abläufe als

Programme dargestellt werden. Ein Programm besteht aus einer Eingabe, mehreren Sub-Aufgaben und einer abschließenden Antwort. Die Sub-Aufgaben können dabei Werkzeugaufrufe enthalten. Wird bei der Generierung ein Werkzeugaufruf erreicht, wird die Generierung unterbrochen, das Werkzeug ausgeführt und anschließend mit dessen Ergebnis fortgesetzt [1].

Dadurch können die Ergebnisse vorheriger Schritte unmittelbar in nachfolgenden Schritten verwendet werden. In einem von ART dargestellten Beispiel wird zunächst eine Suchanfrage ausgeführt, deren Ergebnis anschließend als Grundlage für die Generierung von Code dient. Der generierte Code wird wiederum durch ein weiteres Werkzeug ausgeführt. Die einzelnen Schritte bilden damit eine zusammenhängende Ausführungskette, in der die Ergebnisse vorheriger Werkzeuge den weiteren Ablauf beeinflussen [1].

Eine explizitere Darstellung solcher Abhängigkeiten verwenden Lu et al. im Rahmen von OrchDAG. Ein Werkzeugausführungsplan wird als DAG $G=(V,E)$ dargestellt. Die Knoten repräsentieren dabei Werkzeugaufrufe, während die Kanten Abhängigkeiten zwischen den Aufrufen darstellen. Der Plan enthält für jeden Schritt unter anderem das verwendete Werkzeug, dessen Parameter und die Abhängigkeiten zu vorherigen Schritten. Dadurch kann beispielsweise festgelegt werden, dass ein bestimmtes Eingabefeld aus einem Ergebnis eines vorherigen Werkzeugaufrufs übernommen werden muss [18].

Die Bedeutung solcher Abhängigkeiten zeigt sich insbesondere bei komplexeren Aufgaben. Lu et al. weisen darauf hin, dass in industriellen Anwendungen eine große Anzahl von Domänenwerkzeugen zur Verfügung stehen kann und die Komplexität unter anderem daraus entsteht, dass Werkzeugausgaben viele Felder enthalten und ein Ausgabefeld eines Werkzeugs als Eingabe eines anderen Werkzeugs dienen kann. Zusätzlich können sich Abhängigkeiten über mehrere Interaktionsrunden erstrecken [18].

3.4.2. Planung und Ausführung

Für die Organisation solcher mehrstufigen Abläufe lassen sich unterschiedliche Vorgehensweisen unterscheiden. Zhe et al. ordnen bestehende Ansätze unter anderem zwei Paradigmen zu. Bei reaktiven Verfahren werden Entscheidungen schrittweise während der Ausführung getroffen. Demgegenüber erzeugen Plan-and-Execute-Verfahren zunächst einen globalen Plan, der anschließend ausgeführt wird. Nach Darstellung von Zhe et al. bieten reaktive Verfahren Flexibilität, verfügen jedoch nicht notwendigerweise über eine globale Sicht auf Abhängigkeiten. Vollständig vorab erzeugte Pläne können dagegen empfindlich gegenüber Abweichungen während der tatsächlichen Ausführung sein [19].

Das in [20] beschriebene System stellt ein Beispiel für eine stärker strukturierte, iterative Ausführung dar. Die Architektur trennt dabei drei Aufgabenbereiche: Ein Planner analysiert die Anfrage und erstellt eine Liste auszuführender Aktionen, ein Dispatcher übersetzt diese Aktionen in konkrete strukturierte Werkzeugaufrufe und ein Synthesizer erzeugt abschließend die Antwort. Die Werkzeugergebnisse werden anschließend wieder an den Planner zurückgegeben, der deren Relevanz und Vollständigkeit bewertet und gegebenenfalls weitere Aktionen plant [20].

Damit entsteht eine iterative Schleife aus Planung, Ausführung und anschließender Bewertung der Ergebnisse. Der Planner kann nach einem Werkzeugaufruf entscheiden, ob weitere Aktionen erforderlich sind, ob die vorhandenen Informationen für die Beantwortung ausreichen oder ob ein erneuter bzw. angepasster Aufruf notwendig ist. Das System unterstützt außerdem parallele Werkzeugaufrufe, wenn die geplanten Aktionen unabhängig voneinander ausgeführt werden können [20].

Auch Liu et al. modellieren Orchestrierung als einen iterativen Entscheidungsprozess. Zu jedem Zeitpunkt wird aus mehreren möglichen Aktionen eine Aktion ausgewählt. Zu den möglichen Aktionen gehören unter anderem das direkte Beantworten der Anfrage, Retrieval, Werkzeugnutzung, Verifikation und das Beenden der Ausführung. Die Auswahl erfolgt anhand einer Utility-Funktion, die den erwarteten Nutzen einer Aktion, ihre Kosten, die Unsicherheit und ihre Redundanz berücksichtigt [21].

Dabei wird insbesondere auch die Entscheidung zum Beenden als Teil der Orchestrierung behandelt. Der Ausführungsprozess wird fortgesetzt, bis eine Stop-Aktion ausgewählt wird, ein vorher festgelegtes Schrittbudget erreicht ist oder eine weitere Abbruchbedingung greift. Liu et al. betrachten das Stoppen somit nicht lediglich als technische Randbedingung, sondern als eine eigene Entscheidung innerhalb des Orchestrierungsprozesses [21].

3.4.3. Strukturierung der Werkzeugausführung

Neben der Frage, wann einzelne Werkzeuge aufgerufen werden, kann auch die Struktur der Ausführung vorgegeben oder eingeschränkt werden. Zhe et al. schlagen mit RETO eine Architektur vor, die eine grobe Ausführungsstruktur in Form von Layern verwendet. Werkzeuge werden dabei anhand ihrer vermuteten Abhängigkeiten verschiedenen Ausführungsschichten zugeordnet. Die Werkzeuge werden anschließend schichtweise ausgeführt, sodass das Modell innerhalb eines Schrittes nur auf eine relevante Teilmenge der verfügbaren Werkzeuge zugreifen muss [19].

Die Layer-Struktur stellt dabei keinen vollständig spezifizierten Ausführungsplan dar. Vielmehr soll sie eine grobe globale Struktur bereitstellen, während die konkrete Auswahl und Ausführung innerhalb einer Schicht weiterhin adaptiv erfolgt. RETO trennt damit die globale Organisation der Werkzeugabhängigkeiten von der lokalen Ausführung [19].

Eine andere Form der expliziten Strukturierung findet sich bei ART. Dort wird der Ablauf als strukturiertes Programm mit definierten Sub-Aufgaben und Werkzeugaufrufen dargestellt. Die Autoren argumentieren, dass diese strukturierte Darstellung mehrstufiges Reasoning unterstützt und gegenüber einer freien Generierung von Chain-of-Thought eine zusätzliche Struktur für die Ausführung bereitstellt [1].

OrchDAG geht noch einen Schritt weiter und stellt die Werkzeugausführung explizit als Graph dar. Die Topologie des Graphen beschreibt dabei die Abhängigkeiten zwischen den Werkzeugaufrufen. Die Autoren kontrollieren die Komplexität ihrer generierten Aufgaben unter anderem über Eigenschaften der erzeugten DAGs. Dadurch kann die Komplexität mehrstufiger Tool-Nutzung systematisch variiert werden [18].

3.4.4. Fehler und Anpassung während der Ausführung

Eine besondere Herausforderung entsteht, wenn die tatsächliche Werkzeugausführung von der ursprünglichen Planung abweicht. Lu et al. berücksichtigen in ihrem mehrstufigen Szenario beispielsweise Werkzeugfehler wie Timeouts. In einem solchen Fall kann die weitere Ausführung auf einem Teilgraphen des ursprünglichen Plans basieren. Darüber hinaus berücksichtigen sie Interaktionsrunden, in denen eine neue Anfrage sowohl neue Werkzeuge benötigt als auch von Ergebnissen vorheriger Ausführungen abhängt [18].

Zhe et al. adressieren diese Problematik mit einer lokalen Fehlerkorrektur. RETO überwacht die Werkzeugausführung und versucht fehlerhafte oder inkonsistente Aufrufe innerhalb der bestehenden Layer-Struktur zu korrigieren, anstatt den gesamten Ausführungsplan neu zu erstellen. Dadurch sollen

Fehler auf den betroffenen Werkzeugaufruf begrenzt und deren Ausbreitung auf nachfolgende Schritte reduziert werden [19].

Das von Hernández et al. beschriebene System verfolgt dagegen einen stärker planungsorientierten Ansatz. Der Planner überprüft die Ergebnisse ausgeführter Werkzeuge und kann bei unvollständigen Informationen weitere Aktionen erzeugen oder einen Aufruf erneut beziehungsweise angepasst durchführen. Die Architektur enthält somit ebenfalls eine Form der Anpassung während der Ausführung, wobei die weitere Planung durch die nach jedem Ausführungsschritt gewonnenen Beobachtungen beeinflusst wird [20].

Damit unterscheiden sich die Ansätze insbesondere darin, auf welcher Ebene die Struktur der Ausführung festgelegt wird und wie stark sie während der Ausführung verändert werden kann. Während RETO eine grobe globale Struktur vorgibt und lokale Korrekturen innerhalb dieser Struktur vornimmt, beschreibt [20] einen iterativen Planner, der nach der Ausführung von Werkzeugen weitere Aktionen auf Grundlage der erhaltenen Ergebnisse planen kann. ART wiederum stellt die mehrstufige Ausführung als strukturiertes Programm dar, dessen einzelne Schritte während der Generierung und Werkzeugausführung miteinander verbunden sind [1], [19], [20].

3.4.5. Orchestrierung als Abwägung zwischen Qualität und Ausführungsaufwand

Neben der funktionalen Korrektheit stellt auch der Aufwand der Werkzeugausführung einen Aspekt der Orchestrierung dar. Liu et al. formulieren diesen Zusammenhang explizit als Abwägung zwischen dem erwarteten Nutzen weiterer Aktionen und deren Kosten. Die Utility-Funktion berücksichtigt neben dem erwarteten zusätzlichen Nutzen daher auch einen Kostenfaktor. Zusätzlich werden Unsicherheit und Redundanz berücksichtigt, um weitere Schritte zu fördern, wenn zusätzliche Informationen voraussichtlich hilfreich sind, und unnötige Wiederholungen zu vermeiden [21].

Die Autoren betonen dabei, dass ihre Utility-Funktion keine nachgewiesenermaßen optimale Policy darstellt, sondern als strukturiertes und analysierbares Kontrollverfahren dient. Auch zeigen ihre Experimente nicht, dass der Ansatz grundsätzlich stärkere freie Agenten wie ReAct übertrifft. Der Beitrag wird daher insbesondere als Möglichkeit verstanden, das Verhalten eines Tool-Agents expliziter zu kontrollieren und dessen Kosten- und Entscheidungsverhalten zu analysieren [21].

Auch die anderen betrachteten Ansätze zeigen, dass die Organisation der Tool-Aufrufe mit Effizienzfragen verbunden ist. Hernández et al. berücksichtigen beispielsweise parallele Ausführung für unabhängige Aktionen und formulieren in ihrem Planner explizite Regeln zur Minimierung unnötiger Werkzeugaufrufe und zur frühzeitigen Beendigung, wenn die vorhandenen Informationen als ausreichend angesehen werden [20].

3.4.6. Zusammenfassung

Die betrachteten Arbeiten zeigen, dass Tool-Orchestrierung verschiedene miteinander verbundene Aufgaben umfasst. Dazu gehören insbesondere die Auswahl und Anordnung von Werkzeugaufrufen, die Berücksichtigung von Abhängigkeiten zwischen ihnen sowie die Verarbeitung von Zwischenresultaten. Bei komplexeren Aufgaben kann die Ausführung als strukturierte Kette oder als Graph von Werkzeugaufrufen modelliert werden [1], [18].

Für die Organisation solcher Abläufe lassen sich unterschiedliche Grade der Strukturierung und Anpassbarkeit erkennen. Reaktive Ansätze treffen Entscheidungen schrittweise während der Ausführung, während planbasierte Ansätze zunächst eine übergeordnete Ausführungsstruktur erzeugen. Zwischen diesen Extremen liegen Ansätze wie RETO, die eine grobe globale Struktur mit einer lokal

adaptiven Ausführung verbinden, sowie hierarchische Architekturen, bei denen Planung, technische Ausführung und Synthese getrennt werden [19], [20].

Darüber hinaus ist die Orchestrierung nicht ausschließlich auf die Erzeugung einer möglichst vollständigen Tool-Kette ausgerichtet. Liu et al. zeigen, dass auch die Entscheidung über zusätzliche Schritte, deren erwarteten Nutzen, Redundanz und den Zeitpunkt des Abbruchs Teil der Orchestrierung sein können [21].

Für die vorliegende Arbeit sind damit insbesondere drei Aspekte relevant: die Strukturierung mehrerer Werkzeugaufrufe, die Abhängigkeit und Verarbeitung von Zwischenresultaten sowie der Grad, in dem die Ausführungsentscheidungen durch die Orchestrierungsstrategie vorgegeben oder während der Ausführung durch das LLM getroffen werden. Diese Aspekte bilden die Grundlage für die im Methodikteil untersuchten Orchestrierungsstrategien.

3.5. Evaluation

Die Evaluation von Large Language Models (LLMs) im Zusammenhang mit Tool Use betrachtet unterschiedliche Fähigkeiten, die über die reine Korrektheit einer einzelnen Funktionsausführung hinausgehen. Insbesondere ist zu unterscheiden, ob ein Modell zunächst erkennt, ob ein externes Werkzeug benötigt wird, anschließend das geeignete Werkzeug auswählt und schließlich eine korrekte Folge von Werkzeugaufrufen erzeugt. Die betrachteten Arbeiten setzen dabei unterschiedliche Schwerpunkte und bilden damit verschiedene Ebenen der Tool-Orchestrierung ab.

3.5.1. Entscheidung für die Verwendung von Werkzeugen

Eine grundlegende Voraussetzung für eine sinnvolle Tool-Nutzung ist zunächst die Entscheidung, ob überhaupt ein externes Werkzeug eingesetzt werden sollte. Huang et al. (2024) untersuchen diese Fähigkeit im Rahmen des METATool-Benchmarks. Sie unterscheiden dabei zwischen der awareness of tool usage und der eigentlichen Auswahl von Werkzeugen. Die Autoren begründen diese Unterscheidung damit, dass ein LLM als Agent nicht nur vorhandene Werkzeuge verwenden, sondern zunächst seine eigenen Fähigkeiten einschätzen und entscheiden muss, ob externe Unterstützung erforderlich ist [5].

Für die Evaluation der tool usage awareness verwenden Huang et al. (2024) Accuracy, Precision, Recall und F1-Score. Damit wird insbesondere eine binäre Entscheidung darüber bewertet, ob ein Werkzeug benötigt wird. Für die anschließende Werkzeugauswahl führen die Autoren mit der Correct Selection Rate (CSR) eine eigene Metrik ein. Diese gibt den Anteil der Anfragen an, bei denen die vom Modell ausgewählten Werkzeuge mit den vorgegebenen korrekten Werkzeugen übereinstimmen [5].

METATool berücksichtigt dabei auch Situationen, in denen ein passendes Werkzeug zwar grundsätzlich existiert, jedoch nicht in der zur Verfügung gestellten Werkzeugmenge enthalten ist. In dieser sogenannten tool selection with possible reliability issues-Aufgabe ist das korrekte Verhalten daher nicht die Auswahl eines ähnlichen Werkzeugs, sondern die Vermeidung einer solchen Auswahl. Die Autoren untersuchen damit insbesondere Halluzinationen bei der Werkzeugauswahl [5].

Einen vergleichbaren Schwerpunkt verfolgt WTU-EVAL von Liu et al. (2025). Die Autoren weisen darauf hin, dass viele Untersuchungen Tool Use unter der Annahme evaluieren, dass ein Werkzeug benötigt wird. WTU-EVAL betrachtet dagegen explizit die Frage, ob ein LLM zwischen Situationen unterscheiden kann, in denen ein Werkzeug erforderlich ist, und solchen, in denen die Aufgabe ohne Werkzeug gelöst werden kann. Dafür werden Datensätze, bei denen Werkzeuge benötigt werden,

von allgemeinen Datensätzen unterschieden, bei denen die Aufgaben grundsätzlich ohne externe Werkzeuge lösbar sind [6].

Die Evaluation von WTU-EVAL vergleicht dabei unter anderem Situationen ohne verfügbaren Werkzeugpool mit Situationen, in denen das Modell selbst entscheiden muss, ob es ein Werkzeug verwendet. Die Autoren zeigen, dass eine zusätzliche Werkzeugverfügbarkeit nicht automatisch zu besseren Ergebnissen führt. Insbesondere bei Aufgaben, die eigentlich ohne Werkzeug lösbar sind, kann eine unnötige Werkzeugverwendung die Leistung verschlechtern [6].

Damit zeigen METATOOL und WTU-EVAL eine für die Evaluation von Tool-Orchestrierung relevante Eigenschaft. Ein Toolaufruf ist nicht allein deshalb positiv zu bewerten, weil er formal korrekt ausgeführt wurde. Auch die Entscheidung, kein Werkzeug aufzurufen, kann die korrekte Handlung darstellen.

3.5.2. Auswahl und Anzahl der Werkzeugaufrufe

Neben der Entscheidung für oder gegen Tool Use ist die Auswahl der richtigen Werkzeuge eine weitere zentrale Evaluationsdimension. METATOOL betrachtet hierfür vier unterschiedliche Aufgaben:

- die Auswahl unter ähnlichen Werkzeugen
- die Auswahl in unterschiedlichen Anwendungsszenarien
- die Auswahl unter Bedingungen mit möglichen Zuverlässigkeitsproblemen
- die Auswahl mehrerer Werkzeuge

Die vier Aufgaben sollen unterschiedliche Aspekte der Werkzeugauswahl abbilden, darunter semantisches Verständnis, Anpassungsfähigkeit, Zuverlässigkeit und Schlussfolgerungsfähigkeit [5].

Insbesondere die Aufgabe der Multi-Tool Selection ist für die Orchestrierung relevant. Dabei müssen nicht nur einzelne Werkzeuge identifiziert, sondern mehrere für eine Anfrage relevante Werkzeuge ausgewählt werden. Huang et al. (2024) unterscheiden bei der Auswertung unter anderem zwischen Fällen, in denen beide benötigten Werkzeuge korrekt ausgewählt wurden, nur eines korrekt ausgewählt wurde oder zwei Werkzeuge ausgewählt wurden, von denen nur eines korrekt ist. Dadurch wird sichtbar, dass eine reine Ja/Nein-Bewertung der Werkzeugverwendung bei mehreren benötigten Werkzeugen nur einen Teil der tatsächlichen Leistung erfasst [5].

Eine noch stärker auf Function Calling ausgerichtete Unterteilung verwendet BFCL von Patil et al. (2025). Die Single-Turn-Aufgaben werden danach klassifiziert, wie viele Werkzeuge verfügbar sind und wie viele Funktionsaufrufe erwartet werden. Dabei werden Simple, Multiple, Parallel, Parallel Multiple, Relevance und Irrelevance unterschieden. Während bei Simple ein einzelnes verfügbares Werkzeug einmal aufgerufen wird, enthalten Multiple-Aufgaben mehrere verfügbare Werkzeuge, von denen eines ausgewählt werden muss. Parallel und Parallel Multiple erweitern dies um mehrere gleichzeitig erforderliche Aufrufe. Relevance bezeichnet Situationen, in denen mindestens ein Funktionsaufruf erforderlich ist, während bei Irrelevance trotz vorhandener Werkzeuge kein Aufruf erwartet wird [22].

Diese Kategorisierung ist insbesondere für die Untersuchung von Orchestrierungsstrategien relevant, da sie die Komplexität einer Aufgabe nicht ausschließlich über die Korrektheit eines einzelnen Tool Calls beschreibt. Vielmehr wird zwischen Werkzeugauswahl, mehreren erforderlichen Aufrufen und dem Verzicht auf einen Aufruf unterschieden.

3.5.3. Bewertung einzelner Aufrufe und vollständiger Tool-Sequenzen

Eine wichtige Unterscheidung für die Evaluation komplexerer Tool-Nutzung findet sich bei Lu et al. (2025) im Rahmen von OrchDAG. Dort werden Werkzeugausführungen als gerichtete azyklische

Graphen (DAGs) modelliert. Ein Knoten entspricht dabei einem Werkzeugaufwurf, während Kanten Abhängigkeiten zwischen Werkzeugen darstellen. Ein Ausgabeparameter eines vorherigen Werkzeugaufwurfs kann beispielsweise als Eingabe für einen nachfolgenden Aufruf dienen. Dadurch lässt sich nicht nur bewerten, welche Werkzeuge verwendet werden, sondern auch, wie diese miteinander verbunden sind [18].

Für die Evaluation unterscheiden Lu et al. (2025) explizit zwischen Accuracy/step und Accuracy/user_query. Accuracy/step bewertet jeden einzelnen Handlungsschritt unabhängig. Ein einzelner Schritt kann somit korrekt sein, obwohl die gesamte Werkzeugausführungsstruktur der Anfrage nicht korrekt ist. Accuracy/user_query bewertet dagegen die vollständige Werkzeugausführungsstruktur und ist nur dann korrekt, wenn der gesamte Tool Execution Graph korrekt ist [18].

Diese Unterscheidung zeigt eine wesentliche Herausforderung bei der Bewertung von Tool-Orchestrierung. Eine einzelne korrekte Werkzeugentscheidung und eine vollständig korrekte Lösung einer mehrstufigen Aufgabe sind nicht dasselbe. Ein Modell kann beispielsweise mehrere einzelne Werkzeuge korrekt auswählen, aber eine notwendige Abhängigkeit zwischen ihnen falsch modellieren. Eine reine Bewertung auf Ebene der einzelnen Aufrufe würde einen solchen Fehler nur unvollständig abbilden.

Auch BFCL unterscheidet zwischen unterschiedlichen Ebenen der Evaluation. Für Single-Turn-Aufgaben verwendet BFCL unter anderem eine AST-basierte Bewertung, bei der Funktionsname und Parameter des erzeugten Aufrufs mit den erwarteten Werten abgeglichen werden. Für komplexere Szenarien kommen weitere Evaluationsverfahren zum Einsatz. Bei Multi-Turn-Aufgaben wird beispielsweise sowohl der resultierende Systemzustand als auch die Antwort des Modells überprüft. Ein Eintrag gilt dabei nur dann als korrekt, wenn beide Prüfungen über alle Turns erfolgreich sind [22].

Für parallele Funktionsaufrufe verwendet BFCL außerdem eine All-or-Nothing-Bewertung: Die Reihenfolge paralleler Aufrufe ist zwar irrelevant, jedoch muss jeder erwartete Aufruf gefunden werden. Fehlt auch nur ein erforderlicher Aufruf, gilt die gesamte Prediction als nicht korrekt [22].

3.5.4. Bewertung von Abhängigkeiten von Tools und komplexen Ausführungsplänen

OrchDAG erweitert die Betrachtung von Tool Use insbesondere um Abhängigkeiten zwischen mehreren Aufrufen und um Multi-Turn-Szenarien. Die Autoren berücksichtigen unter anderem Situationen, in denen eine neue Anfrage auf vorherigen Werkzeugergebnissen aufbaut, eine vollständig neue Werkzeugausführungsstruktur benötigt oder ein vorheriger Aufruf aufgrund eines Fehlers erneut ausgeführt werden muss. Die Qualität der erzeugten Tool-Pläne wird dabei anhand ihrer DAG-Struktur überprüft [18].

Die Ergebnisse von OrchDAG verdeutlichen zudem, warum eine getrennte Betrachtung verschiedener Bewertungsebenen sinnvoll ist. In den Experimenten erzielen Modelle teilweise eine deutlich höhere Accuracy/step als Accuracy/user_query. Die Autoren interpretieren dies dahingehend, dass Modelle einzelne Schritte korrekt ausführen können, gleichzeitig aber Schwierigkeiten haben, eine konsistente Gesamtstruktur der Tool-Ausführung aufrechtzuerhalten [18].

Damit ergänzt OrchDAG die stärker auf einzelne Tool Calls ausgerichteten Ansätze von METATOOL, WTU-EVAL und BFCL um die strukturelle Korrektheit einer Tool-Sequenz. Für eine Orchestrierungsstrategie ist diese Ebene besonders relevant, da die Qualität nicht nur davon abhängt, ob einzelne Werkzeuge richtig gewählt werden, sondern auch davon, ob die Gesamtheit der Aufrufe zur Bearbeitung der Aufgabe geeignet ist.

3.5.5. Zusammenfassung der bisherigen Evaluationsansätze

Die betrachteten Arbeiten machen deutlich, dass die Qualität von Tool Use auf mehreren Ebenen bewertet werden kann. METATOOL und WTU-EVAL legen einen Schwerpunkt auf die Entscheidung, ob ein Werkzeug benötigt wird, sowie auf die korrekte Auswahl verfügbarer Werkzeuge. BFCL differenziert stärker nach der Anzahl verfügbarer Werkzeuge und erforderlicher Aufrufe und berücksichtigt dabei auch parallele Aufrufe sowie Situationen, in denen kein Werkzeug verwendet werden soll. OrchDAG betrachtet darüber hinaus die Abhängigkeiten und die Korrektheit vollständiger Tool-Ausführungsstrukturen [5], [22], [18].

Aus diesen Arbeiten lassen sich somit mehrere für die Evaluation von Tool-Orchestrierung relevante Dimensionen ableiten:

- **Tool-Bedarf:** Wird erkannt, ob für eine Aufgabe ein Werkzeug benötigt wird?
- **Tool-Auswahl:** Werden die für die Aufgabe geeigneten Werkzeuge ausgewählt?
- **Aufrufkorrektheit:** Sind Funktionsname und übergebene Parameter korrekt?
- **Vollständigkeit:** Werden alle für die Lösung erforderlichen Werkzeugaufrufe durchgeführt?
- **Redundanz bzw. unnötige Aufrufe:** Werden Werkzeuge aufgerufen, obwohl sie für die Aufgabe nicht benötigt werden?
- **Ausführungsstruktur:** Sind Abhängigkeiten und die Reihenfolge der Werkzeugaufrufe geeignet?
- **Gesamtkorrektheit:** Ist die vollständige Werkzeugausführungssequenz ausreichend, um die gestellte Aufgabe korrekt zu bearbeiten?

Die vorhandenen Arbeiten gewichten diese Dimensionen unterschiedlich. Insbesondere BFCL und OrchDAG zeigen, dass zwischen der Korrektheit einzelner Handlungsschritte und der Korrektheit einer vollständigen Tool-Ausführungsstruktur unterschieden werden kann. OrchDAG macht diese Differenz mit Accuracy/step und Accuracy/user_query explizit, während BFCL bei bestimmten Kategorien eine vollständige Übereinstimmung der erwarteten Aufrufe verlangt [18], [22].

Für die vorliegende Arbeit ergibt sich daraus, dass eine Evaluation von Orchestrierungsstrategien nicht auf eine einzelne Erfolgsmetrik reduziert werden sollte. Während eine Gesamtbewertung der erfolgreichen Aufgabenbearbeitung die praktische Leistungsfähigkeit einer Strategie beschreibt, ermöglicht eine zusätzliche Betrachtung einzelner Tool-Aufrufe die Identifikation der konkreten Ursachen von Fehlentscheidungen. Die genannten Arbeiten liefern damit die Grundlage für eine mehrstufige Evaluation, bei der sowohl die Korrektheit einzelner Toolaufrufe als auch die Qualität der daraus entstehenden gesamten Tool-Ausführung berücksichtigt wird.

3.6. Zwischenfazit

Die Betrachtung des aktuellen Forschungsstands zeigt, dass LLMs durch den Einsatz externer Werkzeuge über die Verarbeitung von Informationen innerhalb des Modells hinaus erweitert werden können. Für den Zugriff auf strukturierte Daten stehen dabei unterschiedliche Ansätze zur Verfügung, darunter die Generierung von Datenbankabfragen sowie der Aufruf bereitgestellter Funktionen. Bei Function Calling werden Werkzeuge und deren erwartete Parameter strukturiert beschrieben, sodass ein Modell geeignete Funktionen auswählen und mit den erforderlichen Argumenten aufrufen kann. Gleichzeitig stellt die Verarbeitung von Zeitreihendaten besondere Anforderungen an die Darstellung und Verarbeitung der Daten, da neben den enthaltenen Werten auch deren zeitliche Struktur berücksichtigt werden muss.

Mit zunehmender Aufgabenkomplexität reicht die Betrachtung einzelner Werkzeugaufrufe jedoch nicht mehr aus. Werden mehrere Werkzeuge benötigt, können Abhängigkeiten zwischen deren

Aufrufen entstehen, sodass Ergebnisse vorheriger Schritte für nachfolgende Aufrufe benötigt werden. Die betrachteten Arbeiten zeigen hierfür unterschiedliche Ansätze zur Planung, Strukturierung und iterativen Ausführung von Werkzeugaufrufen. Diese reichen von strukturierten Programmen und expliziten Ausführungsplänen über graphbasierte Darstellungen von Abhängigkeiten bis hin zu adaptiven Verfahren, bei denen die nächsten Aktionen schrittweise während der Ausführung bestimmt werden. Auch die Berücksichtigung von Ausführungsaufwand, Redundanz und Fehlern kann Bestandteil der Orchestrierung sein.

Für die Bewertung solcher Systeme reicht eine alleinige Betrachtung des finalen Antwortergebnisses ebenfalls nicht aus. Die betrachteten Evaluationsansätze unterscheiden unter anderem zwischen der Entscheidung, ob ein Werkzeug benötigt wird, der Auswahl geeigneter Werkzeuge, der Korrektheit ihrer Parameter sowie der Korrektheit einzelner und vollständiger Werkzeugausführungen. Damit wird deutlich, dass Tool-Nutzung sowohl auf Ebene einzelner Aufrufe als auch auf Ebene einer vollständigen Ausführungssequenz betrachtet werden kann.

Insgesamt ergibt sich aus dem Forschungsstand somit ein Lösungsraum, der verschiedene Möglichkeiten für Datenzugriff, Datenverarbeitung, Tool-Nutzung und deren Orchestrierung umfasst. Die bisherigen Arbeiten betrachten dabei unterschiedliche Formen der Strukturierung und Entscheidungsfreiheit bei der Werkzeugausführung. Welche dieser Möglichkeiten sich für die Verarbeitung von Energiedaten und insbesondere für die Bearbeitung von Aufgaben mit Zeitreihendaten eignen und wie unterschiedliche Orchestrierungsstrategien hinsichtlich ihrer Tool-Nutzung und Effizienz abschneiden, wird im Rahmen der vorliegenden Arbeit untersucht. Die konkrete Auswahl und Ausgestaltung der dafür verwendeten Verfahren sowie deren experimentelle Evaluation werden im folgenden Methodikteil beschrieben.

4. Methodik

4.1. Forschungsdesign

Die Untersuchung wird als vergleichende experimentelle Studie durchgeführt. Ziel ist es, unterschiedliche Ansätze zur Orchestrierung von Tool-Aufrufen unter kontrollierten Bedingungen miteinander zu vergleichen und hinsichtlich ihrer Eignung für die Bearbeitung mehrstufiger Aufgaben zu bewerten. Hierzu werden definierte Testaufgaben verwendet, deren Bearbeitung die Auswahl und gegebenenfalls aufeinander aufbauende Nutzung mehrerer Werkzeuge erfordert.

Für die Vergleichbarkeit der Ergebnisse werden die Versuchsbedingungen soweit möglich konstant gehalten. Insbesondere werden für die zu vergleichenden Ansätze dieselben Testaufgaben, Eingangsdaten und verfügbaren Werkzeuge verwendet. Unterschiede in den Ergebnissen sollen damit möglichst auf die jeweils untersuchte Orchestrierungsstrategie beziehungsweise die betrachtete Modellkonfiguration zurückzuführen sein.

Die verwendeten Testdaten werden kontrolliert beziehungsweise synthetisch erzeugt. Dadurch können relevante Eigenschaften der Aufgaben gezielt variiert und bekannte erwartete Ergebnisse definiert werden. Dies ermöglicht insbesondere die systematische Untersuchung unterschiedlicher Schwierigkeitsgrade und Tool-Abhängigkeiten, ohne von den Eigenschaften eines einzelnen realen Datensatzes abhängig zu sein.

Die Bewertung erfolgt anhand von Kriterien, die sowohl die Korrektheit der Aufgabenbearbeitung als auch die Effizienz der Ausführung berücksichtigen. Neben dem letztendlichen Erfolg einer Aufgabe werden daher auch Eigenschaften der zugrunde liegenden Tool-Aufrufsequenz betrachtet. Dazu gehören insbesondere die korrekte Auswahl und Parametrisierung von Werkzeugen sowie die erfolgreiche Ausführung mehrstufiger Abläufe. Ergänzend werden Laufzeit und gegebenenfalls weitere Ausführungsmerkmale betrachtet.

Vor der eigentlichen Hauptuntersuchung werden ausgewählte technische und methodische Fragestellungen in Voruntersuchungen betrachtet. Diese dienen dazu, geeignete Rahmenbedingungen für die Hauptuntersuchung zu bestimmen und mögliche Einflussfaktoren auf die Ergebnisse zu identifizieren. Aus den Ergebnissen dieser Voruntersuchungen werden, sofern erforderlich, konkrete Designentscheidungen für die anschließenden Experimente abgeleitet.

4.2. Literaturrecherche

Zu Beginn wurde eine strukturierte Literaturrecherche durchgeführt. Hierfür wurden die Datenbanken Google Scholar und IEEE Xplore verwendet. Als Suchbegriffe wurden verschiedene Kombinationen der Begriffe „large language model“, „large language models“, „tool“, „tools“ und „use“ verwendet. Die Ergebnisse wurden jeweils auf Publikationen ab dem Jahr 2022 eingeschränkt. In Google Scholar wurde am 05.03.2026 mit folgendem Suchbegriff gesucht: `allintitle: (tool OR tools) use ("large`

language model" OR "large language models"). Dabei wurden 64 Treffer erzielt. In IEEE Xplore wurde am 10.03.2026 mit folgendem Suchbegriff gesucht: ("Document Title":large language model) AND ("Document Title":tool) AND ("All Metadata":use). Dabei wurden 37 Treffer erzielt. Auf Basis einer ersten Sichtung der Titel und Zusammenfassungen sowie einer anschließenden detaillierten Prüfung der relevanten Publikationen wurden 17 Publikationen ausgewählt, die sich unmittelbar mit Tool Use, Tool Selection oder der Orchestrierung von Large Language Models beschäftigen. Im weiteren Verlauf der Arbeit wurde die Literaturliste durch einzelne zusätzliche Publikationen ergänzt. Diese wurden insbesondere über Referenzen bereits identifizierter Arbeiten sowie durch gezielte Recherchen zu offenen Fragestellungen und thematischen Lücken identifiziert.

4.3. Technische Rahmenbedingungen

Für die Durchführung und Reproduzierbarkeit der Experimente wurde die nachfolgend dargestellte technische Umgebung verwendet. Tabelle 1 gibt einen Überblick über die eingesetzte Hardware und Software einschließlich der jeweiligen Versionen.

Komponente	Spezifikation
Hardware	
GPU	NVIDIA RTX 3090, 24 GB VRAM
CPU	Intel Core i5-13600K
Arbeitsspeicher	32 GB DDR5
Software	
Betriebssystem	Microsoft Windows 11 Pro, 64-Bit, Version 25H2
NVIDIA-Treiber	Version 610.88
Python	Version 3.11.14
Ollama	Version 0.32.9
OpenAI-Python-Paket	Version 2.30.0

Tabelle 1: Technische Rahmenbedingungen

4.3.1. Ausführungsbedingungen

Die Sprachmodelle werden über eine in Python implementierte Schnittstelle angesprochen. Für die lokale Inferenz wird Ollama verwendet. Sowohl die lokalen Modelle als auch das externe API-Modell werden über die OpenAI-Python-Bibliothek angesprochen, wodurch für beide Varianten eine weitgehend einheitliche Schnittstelle verwendet werden kann.

Für die lokalen Modelle wird sichergestellt, dass diese vollständig im GPU-Speicher ausgeführt werden. Dadurch soll CPU-Offloading als zusätzlicher Einflussfaktor auf die Inferenzzeit ausgeschlossen werden [23].

Für die Messungen wurde das Powerlimit der GPU auf 70 % des Standardwerts begrenzt. In vorbereitenden Tests mit identischen Eingaben zeigte sich unter den verwendeten Bedingungen

kein nennenswerter Einfluss dieser Begrenzung auf die Inferenzzeit. Gleichzeitig wurde dadurch die thermische Belastung der GPU reduziert, wodurch mögliche thermisch bedingte Schwankungen der Taktfrequenz begrenzt werden sollten. Diese Einstellung wurde daher für die nachfolgenden Messungen beibehalten [24].

Vor den eigentlichen Messungen werden zunächst Warm-up-Durchläufe durchgeführt, um mögliche Einflüsse der Initialisierung der Inferenzumgebung auf die gemessenen Laufzeiten zu reduzieren.

4.4. Datengrundlage

Für die Evaluation wird eine synthetische Energiedatenbasis verwendet. Die Verwendung synthetischer Daten ermöglicht eine kontrollierte und reproduzierbare Durchführung der Experimente und stellt sicher, dass alle untersuchten Orchestrierungsstrategien auf dieselbe Datengrundlage zugreifen. Die Datenbasis umfasst Messdaten einzelner Zählpunkte sowie Daten, die sich auf die gesamte Energiegemeinschaft beziehen.

4.4.1. Energiedaten

Die Datenbasis umfasst Verbrauchs- und Erzeugungsdaten verschiedener Zählpunkte eines Benutzers. Dabei werden sowohl Zählpunkte mit Verbrauch als auch Zählpunkte mit Erzeugung berücksichtigt. Zusätzlich werden der Gesamtverbrauch und die Gesamterzeugung der Energiegemeinschaft als eigene Datenquellen bereitgestellt.

Die Messdaten werden mit einer zeitlichen Auflösung von 15 Minuten modelliert. Ein Messwert beschreibt dabei die innerhalb eines 15-minütigen Intervalls gemessene Energiemenge in kWh. Zusätzlich werden aktuelle Verbrauchs- und Erzeugungsleistungen in kW bereitgestellt. Der Teilnahmefaktor wird als dimensionsloser Wert zwischen 0 und 1 modelliert und ist für einen Zählpunkt über den gesamten betrachteten Zeitraum konstant.

Die für die Evaluation verwendeten Größen sind in Tabelle 2 zusammengefasst.

Abk.	Bedeutung Englisch	Bedeutung Deutsch	Einheit	Zeitbezug
Basisdaten				
PF	Participation Factor	Teilnahmefaktor	–	konstant
CPC	Current Power Consumption	Aktueller Verbrauch	kW	aktuell
CPG	Current Power Generation	Aktuelle Erzeugung	kW	aktuell
MC	Measured Consumption	Gemessener Verbrauch	kWh	15 min
MG	Measured Generation	Gemessene Erzeugung	kWh	15 min
MC _{EG}	Total Consumption	Gesamtverbrauch	kWh	15 min
MG _{EG}	Total Generation	Gesamterzeugung	kWh	15 min
Abgeleitete Daten				
CP	Community Potential	Gemeinschaftsanteil	kWh	15 min
CC	Community Coverage	Eigenabdeckung	kWh	15 min
WMC	Weighted Measured Consumption	Gemessener Verbrauch gemäß Teilnahmefaktor	kWh	15 min

Abk.	Bedeutung Englisch	Bedeutung Deutsch	Einheit	Zeitbezug
WMG	Weighted Measured Generation	Gemessene Erzeugung gemäß Teilnahmefaktor	kWh	15 min

Tabelle 2: Für die Evaluation verwendete Energiedaten

Die Einteilung in Basisdaten und abgeleitete Daten beschreibt die Verfügbarkeit der jeweiligen Größen innerhalb der Testumgebung. Die Größen MC_{EG} und MG_{EG} könnten grundsätzlich aus den Messwerten der einzelnen Zählpunkte aggregiert werden. Für die vorliegende Evaluation werden sie dennoch als Basisdaten behandelt. Da keine realen Messdaten einer vollständigen Energiegemeinschaft vorliegen, werden sie als synthetische, bereits aggregierte Datenquellen bereitgestellt. Dadurch stehen allen Orchestrierungsstrategien identische und reproduzierbare gemeinschaftsweite Daten zur Verfügung.

4.4.2. Synthetische Datengenerierung

Die benötigten Energiedaten werden synthetisch erzeugt. Für die Verbrauchsdaten wird ein Tagesprofil mit einer Grundlast und typischen Verbrauchsspitzen am Morgen und Abend verwendet. Zusätzlich werden zufällige Schwankungen berücksichtigt. Die Erzeugungsdaten bilden ein vereinfachtes Photovoltaikprofil ab, bei dem während der Nacht keine und während der Tagesstunden eine tageszeitabhängige Erzeugung angenommen wird.

Durch unterschiedliche Skalierungsfaktoren werden Zählpunkte mit unterschiedlichen Verbrauchs- und Erzeugungsniveaus simuliert. Die Energiegemeinschaft wird zusätzlich als virtueller Zählpunkt mit höher skalierten Werten modelliert. Die entsprechenden Zeitreihen stellen somit synthetische gemeinschaftsweite Größen dar und sind nicht als reale Messwerte einer bestehenden Energiegemeinschaft zu interpretieren.

Für die zufälligen Schwankungen wird ein fester Seed verwendet. Dadurch können die Testdaten reproduzierbar erzeugt werden und bleiben bei wiederholter Ausführung mit identischen Parametern unverändert.

Die erzeugten Daten werden in CSV-Dateien gespeichert. Beim Einlesen werden die Zeitstempel in ein standardisiertes Zeitformat überführt und als Index der jeweiligen Zeitreihe verwendet. Dadurch weisen die eingelesenen Zeitreihen eine einheitliche Struktur auf.

4.4.3. Fachliche Definitionen

Die folgenden Definitionen beschreiben die für die Evaluation relevanten Zusammenhänge zwischen den verfügbaren Basisdaten und den daraus abgeleiteten Größen. Dabei bezeichnet t das betrachtete Messintervall, i den Index eines Verbrauchszählpunkts und j den Index eines Erzeugungszählpunkts. n und m bezeichnen die Anzahl der Verbrauchs- bzw. Erzeugungszählpunkte. Der Index k dient als Laufindex bei Summationen über die Verbrauchszählpunkte.

4.4.3.1. Gesamtverbrauch und Gesamterzeugung

Der Gesamtverbrauch und die Gesamterzeugung beschreiben die über alle betrachteten Zählpunkte aggregierten Energiemengen der Energiegemeinschaft.

Der Gesamtverbrauch ergibt sich aus der Summe der gemessenen Verbrauchswerte aller Verbrauchszählpunkte:

$$MC_{EG(t)} = \sum_{i=1}^n MC_{i(t)}$$

Analog ergibt sich die Gesamterzeugung aus der Summe der gemessenen Erzeugungswerte aller Erzeugungszählpunkte:

$$MG_{EG(t)} = \sum_{j=1}^m MG_{j(t)}$$

Die beiden Größen werden in der Testumgebung als Basisdaten bereitgestellt, obwohl sie grundsätzlich aus den Messwerten der einzelnen Zählpunkte aggregiert werden könnten.

4.4.3.2. Teilnahmefaktor und gewichtete Messwerte

Der Teilnahmefaktor bestimmt, mit welchem Anteil der gemessene Verbrauch bzw. die gemessene Erzeugung für die Energiegemeinschaft berücksichtigt wird. Für die Evaluation wird der Teilnahmefaktor als zeitlich konstanter Wert pro Zählpunkt modelliert und als dimensionsloser Faktor zwischen 0 und 1 angegeben.

Der gewichtete Verbrauch eines Verbrauchszählpunkts ergibt sich aus der Multiplikation des gemessenen Verbrauchs mit dem jeweiligen Teilnahmefaktor:

$$WMC_{i(t)} = MC_{i(t)} \cdot PF_{i(t)}$$

Analog ergibt sich die Teilnahmefaktor-gewichtete Erzeugung eines Erzeugungszählpunkts:

$$WMG_{j(t)} = MG_{j(t)} \cdot PF_{j(t)}$$

4.4.3.3. Gemeinschaftsanteil und Eigenabdeckung

Der Gemeinschaftsanteil beschreibt die dem jeweiligen Verbrauchszählpunkt zugeordnete Energiemenge aus der gemeinschaftlichen Erzeugung. Er wird aus der Gesamterzeugung der Energiegemeinschaft und dem Teilnahmefaktor des betrachteten Verbrauchszählpunkts bestimmt:

$$CP_{i(t)} = MG_{EG(t)} \cdot PF_{i(t)}$$

Die Eigenabdeckung begrenzt den Gemeinschaftsanteil auf den tatsächlich für die Energiegemeinschaft berücksichtigten Verbrauch des betrachteten Zählpunkts. Sie entspricht daher dem jeweils kleineren Wert aus Gemeinschaftsanteil und Teilnahmefaktor-gewichteten Verbrauch:

$$CC_{i(t)} = \min(CP_{i(t)}, WMC_{i(t)})$$

Damit kann die Eigenabdeckung weder den verfügbaren Gemeinschaftsanteil noch den Teilnahmefaktor-gewichteten Verbrauch überschreiten.

Die Größen WMC, WMG, CP und CC werden im weiteren Verlauf als abgeleitete Daten bezeichnet.

4.4.4. Datenverfügbarkeit

Für die Evaluation wird ein fester Referenzzeitpunkt verwendet. Die Verfügbarkeit der Energiedaten wird relativ zu diesem Referenzzeitpunkt festgelegt. Die Datenbasis reicht bis einschließlich des Tages, der zwei Kalendertage vor dem Referenzzeitpunkt liegt. Daten des Vortages und des aktuellen Tages gelten somit als nicht verfügbar.

Beispielsweise gilt bei einem Referenzdatum von 02.01.2026:

- verfügbar: bis einschließlich 31.12.2025
- nicht verfügbar: 01.01.2026 und 02.01.2026

Diese Einschränkung wird dem Sprachmodell als Teil der Systembeschreibung mitgeteilt. Ein separater Toolaufruf zur Ermittlung des verfügbaren Datenzeitraums ist daher nicht vorgesehen. Fordert das Sprachmodell dennoch über ein Werkzeug Daten für einen nicht verfügbaren Zeitraum an, wird dies im Rahmen der Evaluation als fehlerhafter Toolaufruf bewertet.

Die Untersuchung der Verarbeitung tatsächlich fehlender oder lückenhafter Messdaten ist nicht Bestandteil der Evaluation. Dadurch bleibt die Datenverfügbarkeit für alle Orchestrierungsstrategien identisch und die Untersuchung konzentriert sich auf die Auswahl und Kombination der verfügbaren Werkzeuge.

4.5. Tooldesign

Das Toolset wurde mit dem Ziel entwickelt, dem Sprachmodell einen kontrollierten Zugriff auf die für die Evaluation relevanten Energiedaten und Berechnungsoperationen zu ermöglichen. Bei der Auswahl der Werkzeuge wurde der Funktionsumfang auf die für die definierten Evaluationsaufgaben erforderlichen Operationen begrenzt. Werkzeuge für externe Datenquellen oder zusätzliche fachliche Bereiche wurden nicht berücksichtigt, da sie den Umfang der Werkzeuglandschaft und die Abhängigkeiten zwischen den einzelnen Werkzeugen erhöhen würden, ohne für die untersuchte Orchestrierung unmittelbar erforderlich zu sein.

4.5.1. Struktur der Werkzeuge

Die verfügbaren Werkzeuge werden als strukturierte Funktionsdefinitionen bereitgestellt. Für jedes Werkzeug werden eine eindeutige Bezeichnung, eine Beschreibung seiner Funktion sowie die erwarteten Parameter und deren Eigenschaften definiert. Die Struktur orientiert sich an der von OpenAI beschriebenen Schnittstelle für Tool-Aufrufe [25]. Auf Grundlage dieser Definitionen kann das Sprachmodell ein geeignetes Werkzeug auswählen und einen strukturierten Aufruf mit den erforderlichen Argumenten erzeugen. Die umgebende Anwendung führt den Aufruf aus und stellt das Ergebnis anschließend wieder dem Sprachmodell zur Verfügung. Die konkrete Implementierung der Werkzeuge bleibt dabei vom Sprachmodell getrennt. Das Modell interagiert somit ausschließlich über die definierten Werkzeugschnittstellen.

Die für die Evaluation bereitgestellten Werkzeuge sind in Tabelle 3 zusammengefasst.

Werkzeug	Funktion
Datenzugriff	
get_participation_factor	Lädt den Teilnahmefaktor eines Zählpunkts.
get_current_power_consumption	Liefert den aktuellen Verbrauch eines Zählpunkts.
get_current_power_generation	Liefert die aktuelle Erzeugung eines Zählpunkts.
get_community_consumption	Lädt den Gesamtverbrauch der Energiegemeinschaft für einen definierten Zeitraum.
get_community_generation	Lädt die Gesamterzeugung der Energiegemeinschaft für einen definierten Zeitraum.

Werkzeug	Funktion
get_measured_energy	Lädt die gemessene Energie eines Zählpunkts für einen definierten Zeitraum.
Allgemeine Operationen	
get_statistical_value	Berechnet einen statistischen Kennwert einer Zeitreihe.
add_timeseries	Addiert mehrere Zeitreihen punktweise.
subtract_timeseries	Subtrahiert eine Zeitreihe punktweise von einer anderen.
multiply_timeseries	Multipliziert eine Zeitreihe mit einem Skalar.
divide_timeseries	Dividiert eine Zeitreihe durch einen Skalar.
min_timeseries	Bestimmt für jedes Messintervall den kleineren Wert mehrerer Zeitreihen.
max_timeseries	Bestimmt für jedes Messintervall den größeren Wert mehrerer Zeitreihen.
Fachliche Berechnungen	
get_weighted_measured_energy	Berechnet die teilnahmefaktor-gewichtete gemessene Energie eines Zählpunkts.
calculate_community_potential	Berechnet den Gemeinschaftsanteil aus der gemeinschaftlichen Erzeugung und dem Teilnahmefaktor.
calculate_community_coverage	Berechnet die Eigenabdeckung eines Verbrauchszählpunkts.
Darstellung	
create_energy_plot	Erzeugt ein Liniendiagramm aus einer oder mehreren Zeitreihen und stellt diese mit einer Legende dar.
Antwortgenerierung	
generate_answer	Erzeugt anhand ausgewählter Ergebnisse und einer frei definierbaren Nachricht eine finale Antwort.

Tabelle 3: Für die Evaluation bereitgestellte Werkzeuge

4.5.2. Zeitliche Parametrisierung

Für die zeitliche Interpretation der Testfälle wird ein fester Referenzzeitpunkt definiert. Dieser umfasst Datum, Uhrzeit und Zeitzone und wird dem Sprachmodell als aktueller Zeitpunkt vorgegeben. Dadurch können sowohl absolute als auch relative Zeitangaben eindeutig in konkrete Zeitintervalle überführt werden. Die Verwendung eines festen Referenzzeitpunkts stellt sicher, dass identische Anfragen unabhängig vom tatsächlichen Zeitpunkt der Versuchsdurchführung auf dieselben Zeitintervalle und damit auf identische Toolparameter abgebildet werden. Zeitintervalle werden durch einen Start- und Endzeitpunkt beschrieben, wobei der Startzeitpunkt inkludiert und der Endzeitpunkt exkludiert wird. Ein einzelner Kalendertag wird daher durch den Beginn dieses Tages und den Beginn des Folgetages beschrieben.

Zur Interpretation natürlicher Zeitangaben wurde zunächst die Verwendung eines separaten Werkzeugs zur Zeitauflösung betrachtet. Eine Voruntersuchung mit aktuellen Sprachmodellen zeigte jedoch, dass die verwendeten Modelle relative und komplexe Zeitangaben mit geeigneten Instruktionen

nen zuverlässig in konkrete Zeitintervalle überführen können. Auf ein separates Zeitauflösungswerkzeug wurde daher verzichtet.

4.5.3. Datenzugriff

Die Datenzugriffswerkzeuge kapseln die konkrete Implementierung des Zugriffs auf die Energiedaten und stellen dem Sprachmodell ausschließlich die in Tabelle 3 definierten Datenzugriffe zur Verfügung. Die abgerufenen Zeitreihen werden in einer einheitlichen Struktur bereitgestellt und enthalten ausschließlich den Zeitstempel und den zugehörigen Energiewert in kWh. Die für die Auswahl der Datenquelle erforderlichen Informationen, insbesondere Zählpunkt und Energieart, werden über die Parameter des jeweiligen Werkzeugs festgelegt und sind nicht Bestandteil der zurückgegebenen Zeitreihe.

4.5.4. Verarbeitung von Zeitreihen

Zeitreihen werden nicht als vollständige numerische Daten direkt an das Sprachmodell übergeben. Der Zugriff auf Zeitreihen und deren numerische Verarbeitung erfolgt ausschließlich über die bereitgestellten Werkzeuge. Für die Darstellung von Zeitreihen steht zusätzlich das Werkzeug `create_energy_plot` zur Verfügung. Dieses kann eine oder mehrere bereits vorliegende Zeitreihen in einem Liniendiagramm darstellen und die einzelnen Zeitreihen anhand der übergebenen Bezeichnungen in einer Legende unterscheiden. Dadurch kann das Sprachmodell neben der Auswahl und Kombination von Datenzugriffs- und Berechnungswerkzeugen auch eine explizit angeforderte Visualisierung veranlassen.

4.5.5. Allgemeine Operationen

Die allgemeinen Operationen stellen elementare Verarbeitungsschritte für Zeitreihen bereit. Sie ermöglichen die punktweise Addition, Subtraktion, Multiplikation und Division sowie die Bestimmung des Minimums und Maximums mehrerer Zeitreihen. Zusätzlich können mit einem eigenen Werkzeug statistische Kennwerte einer Zeitreihe berechnet werden. Die Operationen arbeiten auf Zeitreihen mit identischer zeitlicher Struktur. Die Zeitstempel werden dabei als gemeinsamer Index verwendet, sodass die Berechnung jeweils für dasselbe Messintervall erfolgt.

4.5.6. Fachliche Berechnungen

Die fachlichen Berechnungen stellen höher abstrahierte domänenspezifische Funktionen bereit. Sie kapseln mehrere Datenzugriffe und Berechnungsschritte und ermöglichen dadurch die direkte Berechnung fachlicher Größen der Energiegemeinschaft. Dazu gehören die teilnahmefaktor-gewichtete gemessene Energie, der Gemeinschaftsanteil und die Eigenabdeckung. Die Unterscheidung zwischen allgemeinen Operationen und fachlichen Berechnungen bildet eine wesentliche Grundlage für den Vergleich der Orchestrierungsstrategien. Während höher abstrahierte Werkzeuge komplexere Berechnungsschritte direkt bereitstellen, können dieselben Berechnungen bei Verwendung elementarer Werkzeuge aus mehreren aufeinanderfolgenden Toolaufrufen zusammengesetzt werden.

4.5.7. Antwortgenerierung

Nach Abschluss der Orchestrierung werden die für die Beantwortung der Benutzeranfrage relevanten Ergebnisse an die Funktion `generate_answer` übergeben. Diese erzeugt auf Grundlage der Benutzeranfrage und der bereitgestellten Ergebnisse die finale sprachliche Antwort. Die Auswahl der an `generate_answer` übergebenen Ergebnisse erfolgt abhängig von der jeweiligen Orchestrierungsstrategie. Bei der pipelinebasierten Orchestrierung ist bereits bei der Definition der Pipeline festgelegt,

welches Ergebnis für die finale Antwort verwendet wird. Bei der planbasierten Orchestrierung wird diese Auswahl gemeinsam mit dem Ablaufplan durch das Sprachmodell festgelegt. Bei der iterativen Orchestrierung entsteht die Auswahl erst während der schrittweisen Bearbeitung der Anfrage, da die benötigten Ergebnisse zu Beginn noch nicht vollständig bekannt sind.

Die einzelnen Ergebnisse werden innerhalb der Anwendung über eindeutige Ergebniskennungen referenziert. Dadurch können Zwischenergebnisse unabhängig von ihrer Position innerhalb des Orchestrierungsablaufs eindeutig adressiert und für nachfolgende Werkzeugaufrufe oder die finale Antwort verwendet werden. Die Ergebniskennungen werden von der umgebenden Anwendung vergeben und sind unabhängig von der verwendeten Orchestrierungsstrategie.

4.6. Orchestrierungsstrategien

Die Nutzung mehrerer Werkzeuge stellt nicht nur die Frage nach dem jeweils geeigneten Werkzeug, sondern auch nach der Organisation der einzelnen Werkzeugaufrufe. Sobald mehrere Werkzeuge innerhalb einer Aufgabe verwendet werden, können Abhängigkeiten zwischen den einzelnen Aufrufen entstehen. So kann beispielsweise die Ausgabe eines Werkzeugs als Eingabe für ein nachfolgendes Werkzeug benötigt werden. Die Auswahl der Werkzeuge muss daher gemeinsam mit ihrer Anordnung und Ausführungsreihenfolge betrachtet werden [18].

In dieser Arbeit wird diese Organisation als Orchestrierung bezeichnet. Orchestrierung umfasst damit insbesondere die Auswahl und Anordnung von Werkzeugaufrufen, die Berücksichtigung von Abhängigkeiten sowie die Verarbeitung von Zwischenresultaten. Bei komplexeren Aufgaben kann eine solche Ausführung als strukturierte Kette oder als Graph von Werkzeugaufrufen dargestellt werden [1], [18].

Die betrachteten Ansätze unterscheiden sich insbesondere darin, wie stark die Ausführungsstruktur vorgegeben ist und zu welchem Zeitpunkt das LLM über den weiteren Ablauf entscheidet. In der Forschung werden unter anderem reaktive Verfahren, bei denen Entscheidungen schrittweise während der Ausführung getroffen werden, und planbasierte Verfahren, bei denen zunächst ein globaler Plan erzeugt wird, unterschieden [19]. Auch die in dieser Arbeit betrachtete iterative Strategie basiert auf einer solchen schrittweisen Entscheidung unter Berücksichtigung bereits ausgeführter Werkzeugaufrufe [20].

Für die Untersuchung werden drei Orchestrierungsstrategien betrachtet: eine deterministische, eine planbasierte und eine iterative Strategie. Abbildung 1 stellt die grundlegende Struktur der drei Ansätze gegenüber.

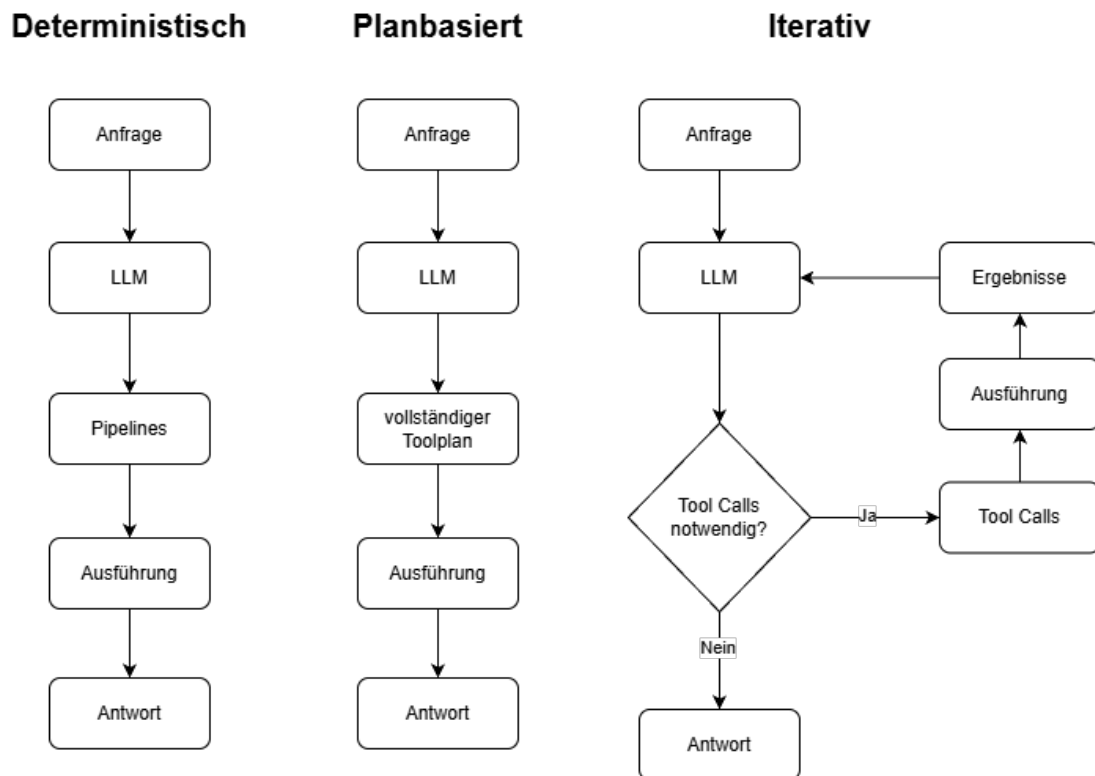


Abbildung 1: Vergleich der untersuchten Orchestrierungsstrategien

Die Abbildung verdeutlicht dabei insbesondere die zunehmende Flexibilität der Ausführungsentscheidung. Bei der deterministischen Strategie wird eine vorgegebene Pipeline ausgewählt und anschließend abgearbeitet. Bei der planbasierten Strategie erzeugt das LLM zunächst einen vollständigen Plan, dessen Schritte anschließend ausgeführt werden. Bei der iterativen Strategie entscheidet das LLM dagegen nach der Ausführung von Werkzeugen erneut über die benötigten nächsten Schritte. Die Strategien unterscheiden sich damit vor allem darin, auf welcher Ebene und zu welchem Zeitpunkt die Ausführungsentscheidungen getroffen werden.

4.6.1. Deterministische Orchestrierung

Die deterministische Orchestrierung stellt den einfachsten der betrachteten Ansätze dar. Die möglichen Pipelines und deren Abläufe sind vorgegeben. Das LLM erhält die Benutzeranfrage sowie die verfügbaren Werkzeuge und entscheidet, welche der vorgesehenen Pipelines für die Anfrage geeignet ist. Die ausgewählte Pipeline wird anschließend entsprechend ihrer festgelegten Reihenfolge abgearbeitet.

Die zentrale Eigenschaft dieses Ansatzes besteht darin, dass die Ausführungsstruktur nach der Auswahl der Pipeline nicht mehr durch das LLM verändert wird. Die Reihenfolge der Werkzeugaufrufe sowie die grundsätzliche Struktur des Ablaufs sind damit durch die jeweilige Pipeline bestimmt. Entscheidungen über alternative Ausführungswege finden während der Abarbeitung nicht statt.

Für die vorliegende Untersuchung ist insbesondere die Auswahl der richtigen Pipeline relevant. Die tatsächliche Ausführung der Werkzeuge ist hierfür nicht erforderlich. Entscheidend ist, ob das LLM anhand der Benutzeranfrage die Pipeline auswählt, die die für die Aufgabe erforderlichen Schritte enthält. Die Pipeline kann anschließend entsprechend der vorgegebenen Struktur ausgeführt werden.

Die deterministische Strategie bildet damit den Fall mit der stärksten Vorabstrukturierung. Die Ausführungsentscheidung des LLM beschränkt sich auf die Auswahl der geeigneten Pipeline, während die einzelnen Ausführungsschritte anschließend durch die vorgegebene Struktur bestimmt werden. Dies ermöglicht eine einfache und nachvollziehbare Ausführung, begrenzt jedoch gleichzeitig die Möglichkeit, den Ablauf an während der Ausführung gewonnene Informationen anzupassen.

4.6.2. Planbasierte Orchestrierung

Bei der planbasierten Orchestrierung wird der konkrete Ablauf nicht mehr durch eine vorgegebene Pipeline festgelegt. Stattdessen erzeugt das LLM zunächst einen vollständigen Plan für die Bearbeitung der Benutzeranfrage. Dieser Plan enthält die benötigten Werkzeugaufrufe und deren Reihenfolge.

Das Prinzip entspricht der in der Forschung beschriebenen Plan-and-Execute-Struktur, bei der zunächst ein globaler Plan erzeugt und anschließend ausgeführt wird [19]. Eine explizite Darstellung von Werkzeugabhängigkeiten findet sich beispielsweise bei OrchDAG. Dort werden Werkzeugaufrufe als Knoten eines gerichteten azyklischen Graphen dargestellt, während Kanten die Abhängigkeiten zwischen den Aufrufen beschreiben [18].

Für die vorliegende Implementierung wird der Plan als Folge von Werkzeugaufrufen dargestellt. Jeder Werkzeugaufruf erhält eine eindeutige ID. Diese entspricht bei dieser Methode seiner Position innerhalb des erzeugten Plans und ermöglicht die Referenzierung des zugehörigen Ergebnisses.

Die tatsächliche Ausführung des Plans ist für die Evaluation der Orchestrierungsentscheidung nicht erforderlich. Bewertet wird, ob das LLM die für die Anfrage erforderlichen Werkzeuge auswählt und diese in der richtigen Reihenfolge in den Plan aufnimmt. Der erzeugte Plan könnte anschließend Schritt für Schritt abgearbeitet werden. Durch die Referenzierung vorheriger Ergebnisse kann dabei festgelegt werden, welches Ergebnis eines vorherigen Werkzeugaufrufs als Eingabe für einen nachfolgenden Aufruf dient.

Im Unterschied zur deterministischen Strategie muss damit nicht bereits vor der Anfrage festgelegt sein, welche konkrete Pipeline für die Aufgabe benötigt wird. Das LLM kann den Ablauf an die jeweilige Anfrage anpassen. Die wesentliche Entscheidung wird jedoch weiterhin vor der Ausführung der geplanten Werkzeugaufrufe getroffen.

Diese Eigenschaft unterscheidet die planbasierte von einer iterativen Orchestrierung. Bei einem vorab erzeugten Plan stehen die späteren Werkzeugergebnisse zum Zeitpunkt der Planung noch nicht zur Verfügung. Zhe et al. beschreiben entsprechend, dass vollständig vorab erzeugte Pläne gegenüber Abweichungen während der tatsächlichen Ausführung empfindlich sein können [19].

4.6.3. Iterative Orchestrierung

Die iterative Orchestrierung legt den konkreten Ausführungsweg nichtvollständig im Voraus fest. Stattdessen entscheidet das LLM schrittweise, welche Werkzeugaufrufe als Nächstes erforderlich sind. Die Ergebnisse bereits ausgeführter Werkzeuge werden dabei in die nächste Entscheidungsrunde einbezogen.

Dieses Vorgehen entspricht grundsätzlich dem in der Forschung beschriebenen Zusammenspiel von Planung, Aktion und Beobachtung. Beim ReAct-Prinzip werden die Ergebnisse einer Aktion als Beobachtung verwendet und können die anschließende Entscheidung beeinflussen [20]. Hernández et al. beschreiben hierzu eine Architektur, in der ein Planner zunächst Aktionen bestimmt, diese ausgeführt werden und die erhaltenen Ergebnisse anschließend wieder an den Planner zurückgegeben werden. Auf Grundlage dieser Ergebnisse können weitere Aktionen geplant werden [20].

Die iterative Strategie der vorliegenden Arbeit folgt diesem Grundprinzip. Das LLM erstellt keinen vollständigen Plan für die gesamte Aufgabe. Statt dessen erzeugt es in jeder Iteration die für den nächsten Ausführungsschritt benötigten Werkzeugaufrufe. Die Werkzeugaufrufe werden anschließend ausgeführt und ihre Ergebnisse für die nächste Iteration bereitgestellt. Dadurch kann die Entscheidung über den weiteren Ablauf von den tatsächlich erhaltenen Ergebnissen abhängig gemacht werden.

Jeder Werkzeugaufruf erhält dabei eine eindeutige ID. Bei der iterativen Methode kodiert diese ID zusätzlich die Iteration, in der der Aufruf erfolgt. Nach der Ausführung wird dieselbe ID zur Referenzierung des erzeugten Ergebnisses verwendet. Dadurch können Ergebnisse eindeutig den zugehörigen Werkzeugaufrufen zugeordnet und in späteren Iterationen referenziert werden.

Die iterative Verarbeitung wird beendet, sobald das LLM den letzten für die Antwort erforderlichen Schritt erreicht. Hierfür steht das Werkzeug `generate_answer` zur Verfügung. Dieser Schritt entscheidet, welche der bisher erzeugten Ergebnisse für die anschließende Antwortgenerierung benötigt werden. Die Orchestrierung kann damit auch dann beendet werden, wenn nicht sämtliche verfügbaren Werkzeuge verwendet wurden.

Die Zahl der Iterationen ist für die vorliegende Implementierung auf sechs begrenzt. Innerhalb einer Iteration können mehrere Werkzeugaufrufe vorgesehen werden. Die Werkzeugausführung erfolgt anschließend anhand der erzeugten Aufrufliste.

Eine zentrale Eigenschaft der iterativen Strategie ist die Verarbeitung der Zwischenergebnisse. Dabei werden die Ergebnisse abhängig von ihrem Datentyp an das LLM übergeben. Insbesondere Zeitreihen würden den Kontext und die Token stark erhöhen. Das Ziel ist es die Zeitreihen mit zur Verfügung stehenden Tools zu verarbeiten.

4.6.4. Ergebnisaufbereitung

Für alle drei Strategien wird eine gemeinsame grundlegende Werkzeug-Infrastruktur verwendet. Das LLM erhält die Benutzeranfrage, einen System-Prompt, die verfügbaren Werkzeuge mit ihren Definitionen, den Benutzerkontext, zeitliche Rahmenbedingungen, Datenverfügbarkeiten sowie fachliche Zusammenhänge und Berechnungsregeln.

Die Kommunikation mit dem lokalen Modell erfolgt über eine OpenAI-kompatible Schnittstelle zu Ollama. Die Wahl des Modells ist dabei unabhängig von der jeweiligen Orchestrierungsstrategie.

Die Ergebnisse von Werkzeugaufrufen werden in einer einheitlichen Struktur gespeichert. Neben der ID des Ergebnisses werden der verwendete Werkzeugaufruf, dessen Argumente und der Status der Ausführung gespeichert. Bei einer erfolgreichen Ausführung wird zusätzlich das Ergebnis des Werkzeugs abgelegt. Bei einem Fehler wird stattdessen die aufgetretene Fehlermeldung gespeichert.

Für die iterative Orchestrierung werden diese Ergebnisse anschließend für die nächste LLM-Iteration aufbereitet. Dabei werden einfache numerische Ergebnisse wie ganzzahlige oder reelle Werte mit ihrem konkreten Wert bereitgestellt. Zeitreihendaten werden dagegen nicht vollständig an das LLM übergeben. Stattdessen werden der ausgeführte Werkzeugaufruf einschließlich seiner Argumente sowie der Ergebnistyp `timeseries` übergeben. Das LLM erhält damit insbesondere Informationen darüber, welches Werkzeug mit welchen Parametern ausgeführt wurde und dass das Ergebnis eine Zeitreihe darstellt, nicht jedoch die einzelnen Werte der Zeitreihe.

Diese Einschränkung wurde gewählt, um die Übergabe umfangreicher numerischer Zeitreihendaten in den Kontext des LLM zu vermeiden. Wie im Stand der Forschung beschrieben, können längere numerische Zeitreihen einen erheblichen Tokenaufwand verursachen und gleichzeitig Schwierigkeiten

bei der Verarbeitung zeitlicher, dimensionaler oder frequenzbezogener Merkmale hervorrufen [16]. Liu et al. zeigen für ein untersuchtes Zeitreihenbeispiel, dass die numerische Repräsentation bis zu 60.000 Input-Tokens umfassen kann. Gleichzeitig untersuchen sie mit VL-Time eine alternative visuelle Repräsentation, die den benötigten Kontext deutlich reduziert [16].

Die in dieser Arbeit gewählte Ergebnisaufbereitung verfolgt einen anderen Ansatz. Die Zeitreihe selbst bleibt in der Werkzeug- beziehungsweise Ausführungsschicht verfügbar, wird jedoch nicht als vollständige numerische Folge in den Kontext des LLM aufgenommen. Dadurch kann das LLM weiterhin erkennen, dass ein entsprechender Werkzeugaufruf erfolgreich eine Zeitreihe bereitgestellt hat und auf welchen Zeitraum sich der Aufruf bezieht. Die Verarbeitung der eigentlichen Zeitreihendaten erfolgt dagegen außerhalb des LLM-Kontexts.

Auch andere nicht-numerische oder umfangreiche Ergebnisse werden nicht zwangsläufig vollständig an das LLM übergeben. Bei erzeugten Plots wird beispielsweise lediglich der Ergebnistyp `plot` als Information für die weitere Orchestrierung bereitgestellt. Die eigentliche Darstellung wird damit nicht Bestandteil des textuellen Kontexts der nächsten Orchestrierungsiteration.

4.6.5. Gemeinsame Konfiguration und Messgrößen

Um die drei Strategien möglichst vergleichbar zu untersuchen, werden weitgehend identische System-Prompts verwendet. Die allgemeinen Informationen und die grundlegende Struktur der Werkzeugbeschreibungen bleiben über die Methoden hinweg gleich. Unterschiede bestehen dort, wo sie für die jeweilige Orchestrierungsstrategie erforderlich sind. Die deterministische Strategie verwendet aufgrund der vorgegebenen Pipelines eine entsprechend angepasste Werkzeugbeschreibung.

Für jeden LLM-Aufruf werden die Laufzeit, die Anzahl der Input-Tokens, die Anzahl der Output-Tokens, die Gesamtzahl der Tokens sowie die Finish Reason erfasst. Für die LLM-Aufrufe wird ein Timeout von 120 Sekunden verwendet. Bei der iterativen Strategie werden diese Werte zusätzlich für jede Iteration separat gespeichert und anschließend zu einer Gesamtnutzung aggregiert.

Die drei Strategien unterscheiden sich damit gezielt in der Organisation der Werkzeugausführung, während die übrige technische Grundlage möglichst vergleichbar gehalten wird. Die deterministische Strategie bildet dabei den Fall einer vorgegebenen Ausführungsstruktur, die planbasierte Strategie einen durch das LLM erzeugten Ablauf und die iterative Strategie eine schrittweise, auf Zwischenergebnissen basierende Orchestrierung.

4.7. Modellauswahl

Für die Evaluation werden drei Modelle ausgewählt, die unterschiedliche Voraussetzungen hinsichtlich lokaler Ressourcen und Modellbereitstellung repräsentieren. Ziel der Auswahl ist dabei nicht, die gesamte Größenabstufung einer Modellfamilie zu untersuchen, sondern den Einfluss unterschiedlicher Modellcharakteristika auf die Wirksamkeit der untersuchten Orchestrierungsmethoden zu betrachten. Hierzu werden mit Qwen3:8B und Qwen3:30B zwei lokal ausführbare Modelle derselben Modellfamilie sowie mit GPT-5.4 mini ein über eine API bereitgestelltes Cloud-Modell verwendet. Die wesentlichen technischen Merkmale der Modelle sind in Tabelle 4 dargestellt.

Merkmale	qwen3:8b	qwen3:30b	gpt-5.4-mini-2026-03-17
Parameter [Mrd]	8,2	30,5	n.v.
Quantisierung	Q4_K_M	Q4_K_M	n.v.
Speichergröße [GB]	5,2	18	n.v.

Merkmale	qwen3:8b	qwen3:30b	gpt-5.4-mini-2026-03-17
Maximales Kontextfenster	40.960	262.144	400.000
Ausführung	lokal	lokal	API

Tabelle 4: Modellauswahl

Anmerkung: n. v. = nicht veröffentlicht.

4.7.1. Lokale Modelle

Die beiden Qwen3-Modelle wurden ausgewählt, um unterschiedliche Anforderungen an die Ressourcen einer lokalen Ausführungsumgebung abzubilden. Qwen3:8B repräsentiert dabei ein vergleichsweise ressourcenarmes Modell, das aufgrund seiner geringeren Parameterzahl auch auf Systemen mit begrenzten GPU- oder Arbeitsspeicherressourcen betrieben werden kann. Qwen3:30B stellt demgegenüber ein deutlich größeres Modell dar, dessen Ausführung leistungsfähigere Hardware erfordert. Beide Modelle stammen aus derselben Modellfamilie und werden mit derselben Quantisierung ausgeführt. Dadurch kann insbesondere untersucht werden, ob die höhere Modellkapazität des 30B-Modells gegenüber dem 8B-Modell einen zusätzlichen Nutzen bei der Bearbeitung der untersuchten Tool-basierten Aufgaben bietet und in welchem Verhältnis dieser mögliche Vorteil zur Verwendung unterschiedlicher Orchestrierungsmethoden steht.

Die Auswahl stellt bewusst keine vollständige Abdeckung der verfügbaren Qwen3-Modellgrößen dar. Kleinere beziehungsweise zwischen den beiden gewählten Modellen liegende Varianten wie Qwen3:4B und Qwen3:14B wurden nicht in die Evaluation aufgenommen. Ziel der Untersuchung ist nicht die Bestimmung eines Zusammenhangs zwischen Parameterzahl und Leistungsfähigkeit über die gesamte Modellfamilie hinweg. Stattdessen sollen zwei deutlich unterschiedliche lokale Ressourcenszenarien gegenübergestellt werden. Die zusätzliche Untersuchung weiterer Modellgrößen würde den Evaluationsumfang erhöhen, ohne für die zentrale Fragestellung nach dem Einfluss der Orchestrierung einen vergleichbaren zusätzlichen Erkenntnisgewinn zu erwarten.

Ein weiteres Auswahlkriterium für die lokalen Modelle war die vollständige Ausführbarkeit innerhalb des verfügbaren GPU-Speichers. Dadurch können beide Modelle unter vergleichbaren lokalen Bedingungen evaluiert werden, ohne dass für die Verarbeitung auf externe Rechenressourcen zurückgegriffen werden muss.

4.7.2. Cloud-Modell

Als dritter Modelltyp wird GPT-5.4 mini in der versionierten Modellvariante gpt-5.4-mini-2026-03-17 eingesetzt. Damit wird neben den beiden lokalen Modellen ein Cloud-Modell berücksichtigt, dessen Ausführung über eine API erfolgt. GPT-5.4 mini wurde insbesondere aufgrund seiner Eignung für Tool-basierte und agentische Anwendungen ausgewählt. OpenAI positioniert das Modell ausdrücklich für Coding, Computer Use und Subagents und dokumentiert unter anderem die Unterstützung von Function Calling und Structured Outputs. In den vom Hersteller veröffentlichten Evaluationen erreicht GPT-5.4 mini zudem hohe Ergebnisse bei Tool-Calling-Aufgaben. [26]

Die Verwendung eines Cloud-Modells erweitert die Untersuchung über die lokale Qwen3-Modellfamilie hinaus. Dadurch kann untersucht werden, ob die beobachteten Unterschiede zwischen den Orchestrierungsmethoden auch bei einem leistungsfähigen proprietären Modell auftreten. Da für GPT-5.4 mini unter anderem die Anzahl der Modellparameter, die Quantisierung und die Größe der Modellgewichte nicht veröffentlicht werden, werden diese Merkmale in der Tabelle als nicht veröffentlicht gekennzeichnet.

Die Laufzeit des Cloud-Modells ist aufgrund der API-basierten Ausführung zudem nicht unmittelbar mit der Laufzeit der lokal ausgeführten Modelle vergleichbar. Während die lokale Laufzeit maßgeblich durch die verwendete Hardware und die lokale Inferenz beeinflusst wird, können bei der API-Ausführung unter anderem Netzwerkverzögerungen, Serverauslastung, API-Latenzen und weitere infrastrukturelle Faktoren auftreten. Die Laufzeitwerte werden daher entsprechend getrennt betrachtet und nicht als direkter Vergleich der reinen Modellinferenz interpretiert.

4.7.3. Vergleichbarkeit der Evaluation

Um den Einfluss der Modellwahl und der Orchestrierungsmethoden möglichst getrennt betrachten zu können, werden für alle drei Modelle dieselben Aufgaben, Prompts und verfügbaren Werkzeuge verwendet. Ebenso werden sämtliche untersuchten Orchestrierungsmethoden mit jedem Modell auf demselben Aufgabensatz ausgeführt. Unterschiede in den Evaluationsergebnissen können dadurch unter den gegebenen Versuchsbedingungen im Zusammenhang mit dem verwendeten Modell und der jeweiligen Orchestrierungsmethode betrachtet werden.

4.8. Aufgabenauswahl

Für die Evaluation wurde ein Aufgabensatz mit insgesamt 90 Aufgaben erstellt. Dieser umfasst jeweils 30 Aufgaben der drei Aufgabentypen **Direkte Datenabfrage**, **Einzelquellenverarbeitung** und **Mehrquellenverarbeitung**. Die Aufgaben wurden so zusammengestellt, dass das verfügbare Toolset unter den jeweiligen Anforderungen der drei Aufgabentypen möglichst vollständig abgedeckt wird. Die Zuordnung der Aufgaben zu den Aufgabentypen erfolgte vor Beginn der Evaluation manuell anhand der für die Bearbeitung erforderlichen Tool- und Verarbeitungsschritte.

4.8.1. Direkte Datenabfrage

Bei direkten Datenabfragen werden Informationen angefordert, die unmittelbar durch einen oder mehrere einzelne Toolaufrufe ermittelt werden können. Werden mehrere Informationen abgefragt, können dafür auch mehrere Toolaufrufe erforderlich sein. Voraussetzung für die Einordnung in diesen Aufgabentyp ist jedoch, dass keine Abhängigkeit zwischen den einzelnen Toolaufrufen besteht und keine Verarbeitung der Ergebnisse mehrerer Quellen erforderlich ist.

4.8.2. Einzelquellenverarbeitung

Bei der Einzelquellenverarbeitung wird eine einzelne Datenquelle zunächst über ein Tool abgerufen und anschließend durch einen Verarbeitungsschritt in das geforderte Ergebnis überführt. Die Aufgaben umfassen dabei verschiedene Verarbeitungsoperationen und Darstellungsformen, darunter beispielsweise die Berechnung von Summen, Mittelwerten, Minimal- und Maximalwerten sowie die Veränderung und Visualisierung von Zeitreihen. Auch bei diesem Aufgabentyp können mehrere Ergebnisse innerhalb einer Aufgabe gefordert werden, sofern diese jeweils unabhängig voneinander aus einer einzelnen Quelle und einem Verarbeitungsschritt hervorgehen.

4.8.3. Mehrquellenverarbeitung

Die Mehrquellenverarbeitung erfordert die gemeinsame Verarbeitung von Informationen aus mehreren Quellen, um das geforderte Ergebnis zu bestimmen. Hierzu zählen beispielsweise die Addition oder Subtraktion von Verbrauchs- und Erzeugungsdaten sowie die Berechnung von Kennzahlen, bei denen mehrere Datenquellen miteinander verknüpft werden müssen. Ebenso können mehrere Ergebnisse innerhalb einer Aufgabe gefordert werden, sofern für deren Berechnung jeweils mehrere Quellen

gemeinsam verarbeitet werden müssen. Im Vergleich zu den vorherigen Aufgabentypen stehen hierbei insbesondere die Abhängigkeiten zwischen mehreren Toolergebnissen im Mittelpunkt.

Die drei Aufgabentypen unterscheiden sich somit hinsichtlich der für ihre Bearbeitung erforderlichen Tool- und Verarbeitungsschritte. Die Einteilung stellt dabei keine allgemeingültige Bewertung der Schwierigkeit einer Aufgabe dar, sondern dient der systematischen Untersuchung unterschiedlicher Anforderungen an die Orchestrierung.

4.8.4. Aufgabenvarianten und Konsistenz

Bei der Erstellung des Aufgabensatzes wurden teilweise mehrere inhaltlich ähnliche Aufgaben aufgenommen. Diese unterscheiden sich beispielsweise hinsichtlich des verwendeten Zählpunkts, des abgefragten Zeitraums oder der konkreten Formulierung der Anfrage. Die Varianten stellen eigenständige Aufgaben dar und werden entsprechend separat bewertet.

Die Aufnahme ähnlicher Aufgaben basiert auf Beobachtungen aus vorangegangenen Testdurchläufen. Dabei zeigte sich, dass Modelle bei vergleichbaren Anfragen nicht immer konsistent reagieren und dieselbe Art von Aufgabe in unterschiedlichen Durchläufen unterschiedlich bearbeiten können. Durch die Verwendung mehrerer Varianten soll der Einfluss einzelner zufälliger Fehlleistungen auf die aggregierten Evaluationsergebnisse reduziert werden. Eine vollständige Eliminierung dieser Variabilität wird dadurch nicht angestrebt.

4.8.5. Unvollständige und nicht eindeutig beantwortbare Anfragen

Ein Teil der Aufgaben wurde bewusst so gestaltet, dass die Anfrage mit den zum jeweiligen Zeitpunkt verfügbaren Informationen nicht eindeutig bearbeitet werden kann. Hierzu gehören beispielsweise Anfragen, bei denen ein Zählpunkt nicht eindeutig spezifiziert ist, eine angeforderte Eigenschaft für den genannten Zählpunkt nicht vorliegt oder für die Anfrage notwendige Daten fehlen. Ebenso wurden Anfragen aufgenommen, deren Inhalt nicht durch das verfügbare Toolset abgedeckt wird.

Mit diesen Aufgaben wird untersucht, ob das Modell die fehlenden oder widersprüchlichen Informationen erkennt und angemessen darauf reagiert. Ist für eine Bearbeitung eine zusätzliche Information erforderlich, wird eine entsprechende Rückfrage erwartet. Ist die Anfrage mit den verfügbaren Werkzeugen nicht bearbeitbar, soll das Modell dies erkennen, anstatt eigenständig Annahmen zu treffen oder einen nicht passenden Toolaufruf auszuführen. Ein unpassender Toolaufruf beziehungsweise eine darauf basierende falsche Bearbeitung wird als Fehler bewertet.

4.9. Bewertungskriterien

Für den Vergleich der Orchestrierungsmethoden werden mehrere Bewertungskriterien herangezogen. Im Mittelpunkt stehen die beiden Qualitätsdimensionen **Korrektheit** und **Effizienz**. Ergänzend werden die **Fehlerrate**, die **Laufzeit** und der **Tokenverbrauch** erfasst. Die Bewertung erfolgt auf Ebene eines vollständigen Durchlaufs einer Nutzeranfrage und berücksichtigt damit nicht nur einzelne Toolaufrufe, sondern die gesamte für die Bearbeitung erforderliche Toolnutzung.

Die Unterscheidung zwischen der Entscheidung für oder gegen eine Toolnutzung und der Auswahl der benötigten Werkzeuge ist auch in bestehenden Arbeiten zur Evaluation von Tool Use relevant. METATOOL untersucht sowohl die allgemeine Bewusstheit darüber, ob ein Tool benötigt wird, als auch die Auswahl eines oder mehrerer geeigneter Werkzeuge [5]. WTU-EVAL betrachtet ebenfalls explizit Situationen, in denen ein Modell entscheiden muss, ob ein Tool benötigt wird. Dabei werden sowohl Aufgaben betrachtet, die den Einsatz eines Werkzeugs erfordern, als auch Aufgaben, die ohne

Werkzeug beantwortet werden können [6]. Für die vorliegende Untersuchung wird diese Perspektive auf die Bewertung vollständiger Orchestrierungsdurchläufe übertragen.

Kriterium	Bedeutung
Korrektheit	Wurden die für die Aufgabe erforderlichen Tools und Argumente korrekt verwendet und die notwendigen Ergebnisse für die weitere Verarbeitung bereitgestellt?
Effizienz	Wurde die Aufgabe korrekt und ohne unnötige Toolaufrufe oder unnötige Argumente bearbeitet?
Fehlerrate	Anteil der Durchläufe mit einem technischen Fehler oder Timeout.
Laufzeit	LLM-Laufzeit eines Durchlaufs in Sekunden; Toolausführung, Speicherung und sonstige Abläufe außerhalb der LLM-Aufrufe werden nicht berücksichtigt.
Tokenverbrauch	Gesamter Verbrauch an Input- und Outputtokens eines Durchlaufs.

Tabelle 5: Bewertungskriterien

4.9.1. Korrektheit

Die Korrektheit bewertet, ob ein vollständiger Orchestrierungsdurchlauf die für die jeweilige Aufgabe erforderlichen Schritte korrekt ausführt. Dabei wird nicht ausschließlich betrachtet, ob ein einzelner Toolaufruf formal korrekt ist, sondern ob die für die Bearbeitung erforderliche Toolauswahl, die zugehörigen Argumente und die Verarbeitung der daraus resultierenden Informationen korrekt erfolgen. Die Betrachtung vollständiger Toolsequenzen ist insbesondere für Aufgaben mit mehreren aufeinander aufbauenden Toolaufrufen relevant. Auch BFCL unterscheidet bei komplexeren Aufgaben zwischen der Korrektheit einzelner Funktionsaufrufe und der Korrektheit der für eine Anfrage erforderlichen Sequenz [22].

4.9.1.1. Toolauswahlkorrektheit

Die Toolauswahl wird danach bewertet, ob die für die Bearbeitung einer Aufgabe erforderlichen Werkzeuge ausgewählt und aufgerufen werden. Dabei kann eine Aufgabe auch korrekt ohne Toolaufruf bearbeitet werden, wenn für ihre Bearbeitung kein verfügbares Tool erforderlich ist. Die Möglichkeit, dass die korrekte Entscheidung in der Nichtverwendung eines Werkzeugs besteht, wird auch in METATOOL und WTU-EVAL berücksichtigt [5], [6].

Bei Aufgaben, für deren Bearbeitung mehrere Werkzeuge erforderlich sind, müssen sämtliche notwendigen Werkzeuge verwendet werden. Wird ein erforderliches Werkzeug nicht aufgerufen oder ein falsches Werkzeug verwendet, gilt die Toolauswahl als nicht korrekt. Ein zusätzlich verwendetes, für die Bearbeitung nicht notwendiges Werkzeug führt dagegen nicht unmittelbar zu einer inkorrekten Toolauswahl. Ein solcher zusätzlicher Aufruf wird im Rahmen der Effizienz bewertet.

Auch die Reihenfolge von Toolaufrufen kann für die Korrektheit relevant sein. Bestehen Abhängigkeiten zwischen mehreren Aufrufen, muss die Reihenfolge eingehalten werden. Ein nachfolgender Toolaufruf kann beispielsweise auf ein Ergebnis eines vorherigen Aufrufs angewiesen sein. Wird eine solche erforderliche Abhängigkeit durch eine falsche Reihenfolge verletzt, wird der Durchlauf als nicht korrekt bewertet. Die Bewertung vollständiger Werkzeugausführungssequenzen ist auch in bestehenden Ansätzen zur Evaluation komplexerer Toolnutzung vorgesehen [18].

Eine besondere Situation stellt die Rückfrage an den Nutzer dar. Kann der korrekte Toolaufruf aus den vorliegenden Informationen eindeutig abgeleitet werden, wird eine stattdessen gestellte Rückfrage als

korrekt, jedoch nicht effizient bewertet. Ist eine zusätzliche Information dagegen tatsächlich erforderlich, um eine nicht begründbare Spekulation zu vermeiden, gilt die Rückfrage als korrekt und effizient.

4.9.1.2. Argumentkorrektheit

Neben der Auswahl der Werkzeuge wird bewertet, ob die für deren Ausführung erforderlichen Argumente korrekt übergeben werden. Ein Toolaufruf mit falschen Argumentwerten wird als nicht korrekt bewertet. Dies gilt auch dann, wenn das grundsätzlich richtige Tool ausgewählt wurde. Die Bewertung von Funktionsaufrufen umfasst damit neben der Auswahl der Funktion auch die zugehörigen Parameter. Eine entsprechende Berücksichtigung von Funktionsnamen und Parameterwerten findet sich auch bei BFCL [22].

Argumente werden dabei auf ihre inhaltliche Korrektheit bezogen auf die jeweilige Aufgabe bewertet. Werden beispielsweise ein falscher Zeitraum, eine falsche Kennung oder ein anderer für die Aufgabe nicht zutreffender Wert übergeben, gilt die Argumentauswahl als nicht korrekt.

4.9.1.3. Weitergabe von Toolergebnissen

Bei einer mehrstufigen Orchestrierung müssen die für die weitere Verarbeitung erforderlichen Ergebnisse der vorherigen Toolaufrufe korrekt weitergegeben werden. Werden notwendige Ergebnisse nicht an den nachfolgenden Verarbeitungsschritt übermittelt und kann die Aufgabe dadurch nicht korrekt weiterbearbeitet werden, gilt der Durchlauf als nicht korrekt.

Die Bewertung bezieht sich dabei auf die für die Orchestrierung erforderliche Verarbeitung der Toolergebnisse. Die sprachliche Ausgestaltung der finalen Antwort wird nicht als eigenständiges Bewertungskriterium erfasst. Entscheidend ist, dass die für die Beantwortung notwendigen Informationen aus den Toolaufrufen korrekt für die Antwortgenerierung bereitgestellt werden.

4.9.1.4. Aggregation der Korrektheit

Die Korrektheit eines einzelnen Durchlaufs wird binär bewertet. Ein korrekter Durchlauf erhält den Wert 1, ein nicht korrekter Durchlauf den Wert 0. Aus den Bewertungen aller Durchläufe wird anschließend der Mittelwert gebildet. Dieser entspricht dem Anteil der korrekten Durchläufe und wird als Korrektheitsrate angegeben. Ein technischer Fehler oder Timeout führt zu einer nicht korrekten Ausführung und wird entsprechend mit 0 bewertet.

$$\text{Korrektheitsrate} = \frac{\text{Anzahl korrekter Durchläufe}}{\text{Anzahl aller Durchläufe}}$$

4.9.2. Effizienz

Die Effizienz bewertet, ob eine Aufgabe nicht nur korrekt, sondern auch ohne unnötige Toolnutzung bearbeitet wurde. Dabei wird zwischen der Korrektheit und der Effizienz bewusst unterschieden. Ein zusätzlicher oder wiederholter Toolaufruf kann daher zu einem korrekten, aber ineffizienten Durchlauf führen.

Ein Durchlauf wird als effizient bewertet, wenn er korrekt ist und keine für die Aufgabenbearbeitung unnötigen Toolaufrufe oder Argumente verwendet. Der Tokenverbrauch wird nicht unmittelbar als Bestandteil der Effizienzbewertung berücksichtigt. Insbesondere bei iterativen Orchestrierungsmethoden kann ein höherer Tokenverbrauch durch zusätzliche erfolgreiche Verarbeitungsschritte entstehen und stellt daher für sich genommen keinen Beleg für eine ineffiziente Orchestrierung dar.

4.9.2.1. Toolauswahleffizienz

Die Toolauswahleffizienz bewertet, ob ausschließlich die für die Bearbeitung der Aufgabe notwendigen Tools aufgerufen werden. Werden alle erforderlichen Tools korrekt verwendet, zusätzlich jedoch ein nicht benötigtes Tool aufgerufen, bleibt der Durchlauf hinsichtlich der Korrektheit korrekt, wird jedoch als nicht effizient bewertet.

Gleiches gilt für wiederholte Aufrufe eines bereits korrekt ausgeführten Tools, sofern der zusätzliche Aufruf keinen für die Aufgabenbearbeitung erforderlichen Zweck erfüllt.

Wird dagegen ein notwendiges Tool überhaupt nicht aufgerufen oder ein falsches Tool verwendet, ist der Durchlauf sowohl nicht korrekt als auch nicht effizient.

Die Unterscheidung zwischen notwendiger und unnötiger Toolnutzung ist insbesondere deshalb relevant, weil ein Toolaufruf nicht allein aufgrund seiner formalen Korrektheit als sinnvoll bewertet werden kann. WTU-EVAL zeigt beispielsweise, dass eine unangemessene bzw. unnötige Toolnutzung die Leistung eines Modells beeinträchtigen kann [6].

4.9.2.2. Argumenteffizienz

Die Argumenteffizienz bewertet, ob nur die für die jeweilige Aufgabe notwendigen und angemessenen Argumente verwendet werden. Ein Argument kann dabei inhaltlich korrekt sein, aber dennoch über den für die Aufgabe erforderlichen Umfang hinausgehen. Dies wird als ineffizient bewertet. Beispielsweise kann ein unnötig großer abgefragter Zeitraum oder eine unnötig große Menge an für die Antwortgenerierung bereitgestellten Ergebnissen zu einer korrekten, aber ineffizienten Ausführung führen. Davon zu unterscheiden sind tatsächlich falsche Argumente. Wird ein falscher Parameterwert übergeben, ist der Toolaufruf nicht korrekt und damit auch nicht effizient.

4.9.2.3. Rückfragen

Rückfragen werden hinsichtlich ihrer Notwendigkeit bewertet. Kann die erforderliche Toolauswahl aus den vorhandenen Informationen eindeutig abgeleitet werden, stellt eine zusätzliche Rückfrage einen vermeidbaren Verarbeitungsschritt dar. Der Durchlauf wird in diesem Fall als korrekt, aber nicht effizient bewertet. Ist die für eine korrekte Bearbeitung notwendige Information dagegen nicht vorhanden und müsste das Modell ohne Rückfrage spekulieren, stellt die Rückfrage einen erforderlichen Verarbeitungsschritt dar. In diesem Fall wird der Durchlauf als korrekt und effizient bewertet.

4.9.2.4. Aggregation der Effizienz

Auch die Effizienz eines einzelnen Durchlaufs wird binär bewertet. Ein effizienter Durchlauf erhält den Wert 1, ein ineffizienter Durchlauf den Wert 0. Voraussetzung für eine effiziente Bewertung ist zunächst die Korrektheit des Durchlaufs. Ein nicht korrekter Durchlauf kann unabhängig von der Ursache nicht als effizient bewertet werden.

Die Effizienzrate wird analog zur Korrektheitsrate als Mittelwert der binären Einzelbewertungen berechnet:

$$\text{Effizienzrate} = \frac{\text{Anzahl effizienter Durchläufe}}{\text{Anzahl aller Durchläufe}}$$

4.9.3. Fehlerrate

Neben der inhaltlichen Bewertung wird erfasst, ob die technische Ausführung eines Durchlaufs erfolgreich abgeschlossen werden konnte. Als Fehler werden Timeouts, Strukturfehler und sonstige technische Fehler erfasst.

Für jeden LLM-Aufruf gilt ein Timeout von zwei Minuten. Wird dieses Zeitlimit überschritten, wird der betreffende Aufruf abgebrochen und der Durchlauf als Fehler bewertet. Bei der iterativen Methode gilt das Zeitlimit für jeden einzelnen LLM-Aufruf innerhalb einer Iteration. Dadurch kann ein iterativer Durchlauf mehrere Aufrufe mit jeweils einem eigenen Zeitlimit enthalten.

Als Strukturfehler werden Ausgaben bezeichnet, die nicht der für die weitere Verarbeitung vorgegebenen Struktur entsprechen und deshalb nicht von der Orchestrierung verarbeitet werden können. Dazu zählen beispielsweise Ausgaben, bei denen die erwartete Struktur nicht in eine verarbeitbare Datenstruktur überführt oder anschließend nicht korrekt ausgelesen werden kann.

Technische Fehler innerhalb der Orchestrierung werden von Fehlern unterschieden, die durch die vom LLM erzeugte Toolauswahl oder Argumentierung verursacht werden. Tritt beispielsweise aufgrund einer fehlerhaften Implementierung ein technischer Fehler auf, obwohl das LLM einen korrekten Toolaufruf mit korrekten Argumenten erzeugt hat, wird dieser Fehler behoben und der Durchlauf erneut ausgeführt. Ein dadurch verursachter technischer Fehler wird somit nicht als Fehlleistung des LLM bewertet. Führt dagegen ein vom LLM erzeugtes falsches Argument zu einem Fehler bei der Toolausführung, wird dies als Fehler des Durchlaufs gewertet. Fehlerhafte Durchläufe werden als nicht korrekt und nicht effizient bewertet.

Die Fehlerrate wird als Anteil der fehlerhaften Durchläufe an allen durchgeführten Durchläufen berechnet:

$$\text{Fehlerrate} = \frac{\text{Anzahl fehlerhafter Durchläufe}}{\text{Anzahl aller Durchläufe}}$$

4.9.4. Laufzeit

Als Laufzeit wird ausschließlich die Zeit berücksichtigt, die für die Verarbeitung durch das jeweilige Large Language Model benötigt wird. Die Ausführung der Tools, die Speicherung von Ergebnissen sowie sonstige Abläufe außerhalb der LLM-Aufrufe werden nicht berücksichtigt.

Die Laufzeit wird in Sekunden erfasst und für die aggregierte Auswertung als arithmetischer Mittelwert über die jeweils erfassten Durchläufe angegeben.

Bei der Interpretation der Laufzeit ist zu berücksichtigen, dass unterschiedliche Modelle auf unterschiedlichen technischen Infrastrukturen beziehungsweise über unterschiedliche Schnittstellen ausgeführt werden. Die gemessene Laufzeit dient daher primär dem Vergleich der Orchestrierungsmethoden innerhalb der jeweiligen Modellbedingungen. Ein direkter Vergleich der absoluten Laufzeiten verschiedener Modelle ist nur eingeschränkt aussagekräftig.

4.9.5. Tokenverbrauch

Als Tokenverbrauch wird die Summe aus Input- und Outputtokens eines Durchlaufs erfasst. Für die aggregierte Darstellung wird der arithmetische Mittelwert des Gesamtverbrauchs berechnet.

Der Tokenverbrauch wird nicht als eigenständiges Kriterium in die binäre Effizienzbewertung einbezogen. Insbesondere bei der iterativen Methode hängt der Gesamtverbrauch wesentlich von der Anzahl

der durchgeführten Iterationen ab. Mit jeder zusätzlichen Iteration können weitere Informationen in den Kontext aufgenommen werden, wodurch der Umfang der folgenden LLM-Aufrufe steigt.

Ein niedriger Tokenverbrauch ist daher nicht grundsätzlich als besser zu bewerten. Ein Durchlauf, der aufgrund eines frühen Fehlers oder Abbruchs nur wenige Iterationen durchführt, kann beispielsweise deutlich weniger Tokens verbrauchen als ein erfolgreicher Durchlauf, der mehrere Iterationen benötigt. Der Tokenverbrauch wird deshalb als deskriptive Ressourcengröße verwendet und bei der Interpretation der Ergebnisse gemeinsam mit Korrektheit, Effizienz und Fehlerrate betrachtet.

4.10. Versuchsablauf

Die Evaluation wird als vollständige Kombination aus den definierten Aufgaben, den ausgewählten Modellen und den untersuchten Orchestrierungsmethoden durchgeführt. Für jede Kombination aus Modell, Orchestrierungsmethode und Aufgabe wird genau ein Durchlauf ausgeführt. Bei 90 Aufgaben und den drei ausgewählten Modellen wird somit jede Aufgabe unter jeder Orchestrierungsmethode mit jedem Modell bearbeitet. Auf eine mehrfache Ausführung identischer Kombinationen wird aufgrund des hohen daraus resultierenden Evaluationsumfangs verzichtet. Die Berücksichtigung möglicher Inkonsistenzen im Modellverhalten erfolgt stattdessen durch die im Aufgabensatz enthaltenen inhaltlich ähnlichen Aufgabenvarianten.

Zu Beginn eines jeden Durchlaufs wird dem LLM die jeweilige Nutzeranfrage zusammen mit der für die Bearbeitung erforderlichen Systemanweisung und dem verfügbaren Toolset bereitgestellt. Zusätzlich erhält das Modell allgemeine Informationen über den zugrunde liegenden Datensatz sowie die vorhandenen Zählpunkte. Zu den allgemeinen Informationen gehören beispielsweise Einschränkungen hinsichtlich des verfügbaren Datenbestands. Dadurch verfügt das Modell über die für die Planung und Ausführung der jeweiligen Aufgabe relevanten Rahmenbedingungen.

Der weitere Ablauf unterscheidet sich abhängig von der eingesetzten Orchestrierungsmethode. Bei der deterministischen Orchestrierung wird ein einzelner LLM-Aufruf durchgeführt, in dem anhand der Nutzeranfrage die erforderlichen Pipelines ausgewählt werden. Die ausgewählten Pipelines werden anschließend ausgeführt. Welche der daraus resultierenden Toolergebnisse für die abschließende Antwort verwendet werden, ist durch die jeweilige Pipeline festgelegt und wird nicht durch das LLM bestimmt.

Bei der planbasierten Orchestrierung erzeugt das LLM in einem einzelnen Aufruf einen Plan für die Bearbeitung der Aufgabe. Neben den vorgesehenen Verarbeitungsschritten legt der Plan auch fest, welche der erzeugten Toolergebnisse für die anschließende Antwortgenerierung verwendet werden sollen. Die im Plan vorgesehenen Verarbeitungsschritte werden anschließend ausgeführt und die im Plan bestimmten Ergebnisse für die Antwortgenerierung bereitgestellt.

Bei der iterativen Orchestrierung kann die Aufgabe hingegen mehrere LLM-Aufrufe erfordern. Nach der Ausführung der vom LLM vorgeschlagenen Werkzeuge werden die daraus resultierenden Ergebnisse erneut an das LLM übergeben. Auf Grundlage dieser Ergebnisse entscheidet das Modell, ob weitere Toolaufrufe erforderlich sind oder die Aufgabe bereits ausreichend bearbeitet wurde. Dieser Ablauf wird wiederholt, bis das Modell keine weiteren Toolaufrufe mehr anfordert. Anschließend bestimmt das LLM, welche der vorliegenden Toolergebnisse für die Beantwortung der Nutzeranfrage relevant sind und für die abschließende Antwortgenerierung verwendet werden. Zeitreihendaten werden dabei nicht als Rohdaten an das LLM übergeben, sondern entsprechend der im Kapitel zur Orchestrierung beschriebenen Verarbeitung aufbereitet.

Während jedes Durchlaufs werden die Ergebnisse der Modell- und Toolinteraktionen sowie die für die spätere Evaluation relevanten Messwerte protokolliert. Die Erfassung der Laufzeit und des Tokenverbrauchs beschränkt sich dabei ausdrücklich auf die LLM-Aufrufe. Die Laufzeit von Toolaufrufen sowie weitere Verarbeitungsschritte der Orchestrierung werden nicht in diese Messwerte einbezogen. Bei den einstufigen Orchestrierungsmethoden entspricht die gemessene LLM-Laufzeit daher der Dauer des jeweiligen einzelnen LLM-Aufrufs. Beim iterativen Verfahren werden Laufzeit und Tokenverbrauch jedes einzelnen LLM-Aufrufs erfasst und anschließend zu einer Gesamtnutzung (`total_usage`) aggregiert. Der Ablauf entspricht somit konzeptionell einer Folge aus LLM-Aufruf, Toolausführung und erneuter LLM-Verarbeitung, bis keine weiteren Toolaufrufe erforderlich sind und die relevanten Ergebnisse für die abschließende Antwort bestimmt wurden.

Tritt während eines Durchlaufs ein technischer Fehler auf, wird die Bearbeitung der jeweiligen Aufgabe abgebrochen. Der Fehler wird für die spätere manuelle Bewertung protokolliert. Eine erneute Ausführung derselben Kombination aus Aufgabe, Modell und Orchestrierungsmethode erfolgt nicht.

5. Ergebnisse

Im Rahmen der Evaluation wurden insgesamt 810 Durchläufe durchgeführt. Diese ergeben sich aus 90 Aufgaben, die jeweils mit drei Orchestrierungsmethoden und drei Modellen bearbeitet wurden. Die 90 Aufgaben verteilen sich gleichmäßig auf die drei Aufgabentypen direkte Datenabfrage, Einzelquellenverarbeitung und Mehrquellenverarbeitung mit jeweils 30 Aufgaben.

Für die Bewertung werden die Korrektheit und Effizienz der Orchestrierung als zentrale Kriterien betrachtet. Ergänzend werden die Fehlerrate, die durchschnittliche Laufzeit der LLM-Aufrufe sowie der durchschnittliche Tokenverbrauch ausgewertet. Korrektheit und Effizienz werden als Raten angegeben. Laufzeit und Tokenverbrauch stellen Mittelwerte der jeweils erfassten Werte dar.

5.1. Gesamtvergleich der Methoden

Der Vergleich der drei Orchestrierungsmethoden zeigt deutliche Unterschiede hinsichtlich Korrektheit und Effizienz.

Methode	Korrektheit	Effizienz	Laufzeit [s]	Tokens	Fehlerrate
Deterministisch	0,72	0,65	7,67	4019	0.15
Planbasiert	0.59	0.49	8.84	5331	0.29
Iterativ	0.44	0.38	13.16	8273	0.31

Tabelle 6: Gesamtvergleich der Methoden

Die deterministische Methode erreicht mit einer Korrektheitsrate von 0,72 den höchsten Wert der drei Methoden. Die planbasierte Methode erreicht eine Korrektheitsrate von 0,59, während die iterative Methode mit 0,44 den niedrigsten Wert aufweist. Ein vergleichbares Verhältnis zeigt sich bei der Effizienz. Die deterministische Methode erreicht mit 0,65 den höchsten Wert, gefolgt von der planbasierten Methode mit 0,49 und der iterativen Methode mit 0,38.

Auch bei der Laufzeit unterscheiden sich die Methoden. Die deterministische Methode weist mit durchschnittlich 7,67 Sekunden die niedrigste Laufzeit auf. Die planbasierte Methode benötigt durchschnittlich 8,84 Sekunden, während die iterative Methode mit 13,16 Sekunden den höchsten Wert erreicht.

Beim durchschnittlichen Tokenverbrauch zeigt sich ebenfalls eine Abstufung zwischen den Methoden. Die deterministische Methode benötigt durchschnittlich 4.019 Tokens, die planbasierte Methode 5.331 Tokens und die iterative Methode 8.273 Tokens.

Die Fehlerrate beträgt bei der deterministischen Methode 0,15. Die planbasierte Methode weist eine Fehlerrate von 0,29 auf, während die iterative Methode mit 0,31 den höchsten Wert erreicht. Damit weist die deterministische Methode in allen betrachteten Kriterien außer der expliziten Betrachtung von Einzelaspekten der Komponenten die günstigsten Werte auf.

5.2. Analyse der Korrektheits- und Effizienzkomponenten

Zur näheren Betrachtung der Korrektheits- und Effizienzergebnisse werden die zugrunde liegenden Komponenten Toolauswahl und Argumentübergabe getrennt betrachtet.

Methode	Toolauswahl korrekt	Argument korrekt	Toolauswahl effizient	Argumente effizient
Deterministisch	0.75	0.72	0.68	0.65
Planbasiert	0.60	0.59	0.50	0.49
Iterativ	0.45	0.44	0.39	0.38

Tabelle 7: Analyse der Korrektheits- und Effizienzkomponenten

Die Werte der einzelnen Komponenten zeigen, dass die Korrektheit der Toolauswahl bei allen drei Methoden nur geringfügig über der Korrektheit der Argumentübergabe liegt. Bei der deterministischen Methode beträgt die Rate der korrekten Toolauswahl 0,75 und die Rate der korrekten Argumentübergabe 0,72. Bei der planbasierten Methode liegen die entsprechenden Werte bei 0,60 und 0,59. Die iterative Methode erreicht 0,45 bei der Toolauswahl und 0,44 bei den Argumenten.

Ein vergleichbares Verhältnis zeigt sich bei der Effizienz. Die deterministische Methode erreicht eine effiziente Toolauswahl in 0,68 der Fälle und eine effiziente Argumentübergabe in 0,65 der Fälle. Bei der planbasierten Methode liegen die Werte bei 0,50 beziehungsweise 0,49. Die iterative Methode erreicht 0,39 beziehungsweise 0,38.

Damit liegen die Werte der Argumentkomponenten bei allen drei Methoden jeweils nur geringfügig unter den entsprechenden Werten der Toolauswahl. Die größten Unterschiede zwischen den Methoden zeigen sich somit bereits bei der Auswahl der benötigten Tools.

5.3. Gesamtvergleich der Aufgabentypen

Neben dem Vergleich der Orchestrierungsmethoden wird untersucht, wie sich die Ergebnisse in Abhängigkeit vom Aufgabentyp unterscheiden. Da die 90 Aufgaben gleichmäßig auf die drei Aufgabentypen verteilt sind, entfallen jeweils 30 Aufgaben auf direkte Datenabfragen, Einzelquellenverarbeitung und Mehrquellenverarbeitung.

Aufgabentyp	Korrekt-heit	Effizienz	Laufzeit [s]	Tokens	Fehlerrate
Direkte Datenabfrage	0,75	0,72	8,43	6102	0,11
Einzelquellenverarbeitung	0,53	0,44	11,15	5718	0,24
Mehrquellenverarbeitung	0,46	0,36	9,82	5213	0,40

Tabelle 8: Vergleich der Aufgabentypen

Bei den direkten Datenabfragen wird mit 0,75 die höchste Korrektheitsrate erreicht. Die Einzelquellenverarbeitung erreicht eine Korrektheitsrate von 0,53, während die Mehrquellenverarbeitung mit 0,46 den niedrigsten Wert aufweist.

Dasselbe Muster zeigt sich bei der Effizienz. Direkte Datenabfragen erreichen mit 0,72 den höchsten Wert. Die Einzelquellenverarbeitung erreicht 0,44 und die Mehrquellenverarbeitung 0,36.

Die Fehlerrate steigt dagegen von 0,11 bei direkten Datenabfragen über 0,24 bei der Einzelquellenverarbeitung auf 0,40 bei der Mehrquellenverarbeitung. Die niedrigste Fehlerrate tritt damit bei den direkten Datenabfragen auf, während die Mehrquellenverarbeitung die höchste Fehlerrate aufweist.

Bei der durchschnittlichen Laufzeit erreicht die direkte Datenabfrage mit 8,43 Sekunden den niedrigsten Wert. Die Einzelquellenverarbeitung weist mit 11,15 Sekunden die höchste durchschnittliche Laufzeit auf. Die Mehrquellenverarbeitung liegt mit 9,82 Sekunden dazwischen.

Beim durchschnittlichen Tokenverbrauch weist die direkte Datenabfrage mit 6.102 Tokens den höchsten Wert auf. Die Einzelquellenverarbeitung erreicht 5.718 Tokens und die Mehrquellenverarbeitung 5.213 Tokens.

5.4. Vergleich der Modelle

Neben der Orchestrierungsmethode wird der Einfluss des verwendeten Modells betrachtet.

Modell	Korrektheit	Effizienz	Laufzeit [s]	Tokens	Fehlerrate
gpt-5.4-mini	0,73	0,57	2,14	4351	0,00
qwen3:30b	0,46	0,44	11,56	6250	0,50
qwen3:8b	0,56	0,51	18,63	7252	0,25

Tabelle 9: Vergleich der verwendeten Modelle

Beim Vergleich der Modelle erreicht gpt-5.4-mini mit 0,73 die höchste Korrektheitsrate. qwen3:8b erreicht eine Korrektheitsrate von 0,56, während qwen3:30b mit 0,46 den niedrigsten Wert aufweist.

Auch bei der Effizienz erreicht gpt-5.4-mini mit 0,57 den höchsten Wert. qwen3:8b erreicht 0,51 und qwen3:30b 0,44.

Deutliche Unterschiede zeigen sich bei der Fehlerrate. Für gpt-5.4-mini beträgt diese 0,00. qwen3:8b erreicht eine Fehlerrate von 0,25, während qwen3:30b mit 0,50 den höchsten Wert der drei Modelle aufweist.

Auch die durchschnittliche Laufzeit unterscheidet sich deutlich. gpt-5.4-mini weist mit 2,14 Sekunden die niedrigste durchschnittliche Laufzeit auf. qwen3:30b erreicht 11,56 Sekunden und qwen3:8b 18,63 Sekunden.

Beim durchschnittlichen Tokenverbrauch benötigt gpt-5.4-mini mit 4.351 Tokens die geringste Anzahl an Tokens. qwen3:30b erreicht 6.250 Tokens und qwen3:8b 7.252 Tokens.

5.5. Vergleich der Modelle und Methoden

Die bisherigen Vergleiche betrachten Modelle und Methoden jeweils getrennt. Um zu untersuchen, ob sich die Ergebnisse der Methoden abhängig vom verwendeten Modell unterscheiden, werden die Kombinationen aus Modell und Orchestrierungsmethode betrachtet.

Modell	Methode	Korrekt-heit	Effizienz	Laufzeit [s]	Tokens	Fehlerrate
gpt-5.4-mini	deterministic	0,83	0,70	1,82	3204	0,00
	plan-based	0,79	0,58	2,08	4503	0,00
	iterative	0,56	0,42	2,51	5344	0,00

Modell	Methode	Korrekt- heit	Effizienz	Laufzeit [s]	Tokens	Fehlerrate
qwen3:30b	deterministic	0,67	0,64	10,32	4656	0,23
	plan-based	0,30	0,29	12,04	6261	0,68
	iterative	0,40	0,39	13,51	9216	0,59
qwen3:8b	deterministic	0,66	0,61	12,52	4432	0,21
	plan-based	0,67	0,60	16,00	6001	0,20
	iterative	0,36	0,32	29,19	12149	0,34

Tabelle 10: Vergleich der Modelle und Methoden

Die kombinierte Betrachtung von Modell und Methode zeigt, dass die Korrektheit der Methoden innerhalb der Modelle unterschiedlich ausfällt.

Bei gpt-5.4-mini erreicht die deterministische Methode mit 0,83 die höchste Korrektheitsrate. Die planbasierte Methode erreicht 0,79 und die iterative Methode 0,56. Auch hinsichtlich der Effizienz liegt die deterministische Methode mit 0,70 vor der planbasierten Methode mit 0,58 und der iterativen Methode mit 0,42. Die Fehlerrate beträgt bei allen drei Methoden 0,00. Die durchschnittliche Laufzeit liegt zwischen 1,82 Sekunden bei der deterministischen und 2,51 Sekunden bei der iterativen Methode. Der Tokenverbrauch steigt von 3.204 Tokens bei der deterministischen über 4.503 Tokens bei der planbasierten auf 5.344 Tokens bei der iterativen Methode.

Bei qwen3:30b erreicht die deterministische Methode mit 0,67 die höchste Korrektheitsrate. Die iterative Methode erreicht 0,40 und die planbasierte Methode 0,30. Ein ähnliches Verhältnis zeigt sich bei der Effizienz. Die deterministische Methode erreicht 0,64, die iterative Methode 0,39 und die planbasierte Methode 0,29. Die Fehlerrate beträgt bei der deterministischen Methode 0,23, bei der planbasierten Methode 0,68 und bei der iterativen Methode 0,59. Die durchschnittliche Laufzeit beträgt 10,32 Sekunden bei der deterministischen, 12,04 Sekunden bei der planbasierten und 13,51 Sekunden bei der iterativen Methode. Der Tokenverbrauch beträgt 4.656, 6.261 beziehungsweise 9.216 Tokens.

Bei qwen3:8b erreicht die planbasierte Methode mit 0,67 die höchste Korrektheitsrate, knapp vor der deterministischen Methode mit 0,66. Die iterative Methode erreicht 0,36. Auch bei der Effizienz liegt die planbasierte Methode mit 0,60 knapp vor der deterministischen Methode mit 0,61 beziehungsweise die Werte liegen nahezu gleichauf. Die iterative Methode erreicht 0,32. Die Fehlerrate beträgt 0,21 bei der deterministischen, 0,20 bei der planbasierten und 0,34 bei der iterativen Methode. Die durchschnittliche Laufzeit beträgt 12,52 Sekunden bei der deterministischen, 16,00 Sekunden bei der planbasierten und 29,19 Sekunden bei der iterativen Methode. Der Tokenverbrauch beträgt 4.432 Tokens bei der deterministischen, 6.001 Tokens bei der planbasierten und 12.149 Tokens bei der iterativen Methode.

Damit zeigt die kombinierte Betrachtung, dass die Rangfolge der Methoden nicht für alle Modelle identisch ist. Während die deterministische Methode bei gpt-5.4-mini und qwen3:30b die höchste Korrektheitsrate erreicht, weist bei qwen3:8b die planbasierte Methode den höchsten Korrektheitswert auf.

5.6. Fehlerrate

Die Fehlerrate beschreibt den Anteil der Durchläufe, bei denen ein Fehler auftrat. Ein wesentlicher Anteil der beobachteten Fehler entfällt auf Timeouts. Ein LLM-Aufruf wird nach einer Laufzeit von

zwei Minuten abgebrochen und der betreffende Durchlauf als fehlerhaft bewertet. Bei der iterativen Methode gilt dieses Zeitlimit für jeden einzelnen LLM-Aufruf innerhalb einer Iteration.

Über alle 810 Durchläufe trat einmal ein Strukturfehler auf. Dieser trat beim Modell qwen3:8b auf. Dabei wurde eine Ausgabe in einer Form zurückgegeben, die beim anschließenden Zugriff nicht verarbeitet werden konnte. Die übrigen beobachteten Fehler entfielen überwiegend auf Timeouts.

Darüber hinaus traten bei der iterativen Methode fehlerhafte Toolaufrufe auf, bei denen Toolaufrufe einer nachfolgenden Iteration vorweggenommen und dabei unter anderem nicht vorhandene `result_ids` verwendet wurden. Diese Fälle wurden nicht als Strukturfehler erfasst, da die zurückgegebene Struktur grundsätzlich verarbeitet werden konnte. Sie wurden stattdessen als fehlerhafte Toolaufrufe bewertet.

Die Fehlerraten unterscheiden sich sowohl zwischen den Methoden als auch zwischen den Modellen. Auf Methodenebene weist die deterministische Methode mit 0,15 die niedrigste Fehlerrate auf. Die planbasierte Methode erreicht 0,29 und die iterative Methode 0,31. Auf Modellebene weist gpt-5.4-mini mit 0,00 keine Fehler in den betrachteten Durchläufen auf. qwen3:8b erreicht 0,25 und qwen3:30b 0,50.

Die kombinierte Betrachtung von Modell und Methode zeigt ebenfalls deutliche Unterschiede. Bei gpt-5.4-mini beträgt die Fehlerrate unabhängig von der Methode 0,00. Bei qwen3:30b reicht sie von 0,23 bei der deterministischen Methode bis 0,68 bei der planbasierten Methode. Bei qwen3:8b liegen die Werte zwischen 0,20 bei der planbasierten und 0,34 bei der iterativen Methode.

5.7. Laufzeit

Die Laufzeit wird als durchschnittliche Dauer der gemessenen LLM-Aufrufe angegeben. Die Ausführung der Tools sowie weitere Verarbeitungsschritte außerhalb der LLM-Aufrufe sind nicht Bestandteil der Laufzeitmessung. Durchläufe ohne gültigen Laufzeitwert, beispielsweise aufgrund eines Timeouts, werden bei der Berechnung des Mittelwerts nicht berücksichtigt.

Auf Methodenebene weist die deterministische Methode mit durchschnittlich 7,67 Sekunden die niedrigste Laufzeit auf. Die planbasierte Methode erreicht 8,84 Sekunden und die iterative Methode 13,16 Sekunden.

Bei den Modellen unterscheiden sich die durchschnittlichen Laufzeiten stärker. gpt-5.4-mini erreicht 2,14 Sekunden, qwen3:30b 11,56 Sekunden und qwen3:8b 18,63 Sekunden.

Die Kombination von Modell und Methode zeigt, dass die Laufzeit innerhalb eines Modells ebenfalls mit der verwendeten Methode variiert. Bei gpt-5.4-mini liegen die Werte zwischen 1,82 Sekunden bei der deterministischen und 2,51 Sekunden bei der iterativen Methode. Bei qwen3:30b reichen sie von 10,32 Sekunden bei der deterministischen bis 13,51 Sekunden bei der iterativen Methode. Bei qwen3:8b beträgt die Laufzeit 12,52 Sekunden bei der deterministischen, 16,00 Sekunden bei der planbasierten und 29,19 Sekunden bei der iterativen Methode.

Die höchsten Laufzeitwerte treten damit insbesondere bei der Kombination aus qwen3:8b und der iterativen Methode auf.

5.8. Tokenverbrauch

Der Tokenverbrauch wird als durchschnittlicher Gesamtverbrauch von Input- und Output-Tokens ausgewertet.

Auf Methodenebene weist die deterministische Methode mit durchschnittlich 4.019 Tokens den niedrigsten Verbrauch auf. Die planbasierte Methode erreicht 5.331 Tokens, während die iterative Methode mit 8.273 Tokens den höchsten Wert aufweist.

Auch zwischen den Modellen bestehen Unterschiede. gpt-5.4-mini erreicht durchschnittlich 4.351 Tokens, qwen3:30b 6.250 Tokens und qwen3:8b 7.252 Tokens.

Bei der kombinierten Betrachtung von Modell und Methode zeigt sich für alle drei Modelle ein höherer Tokenverbrauch der iterativen Methode gegenüber der deterministischen Methode. Bei gpt-5.4-mini steigt der Verbrauch von 3.204 auf 5.344 Tokens, bei qwen3:30b von 4.656 auf 9.216 Tokens und bei qwen3:8b von 4.432 auf 12.149 Tokens. Die planbasierte Methode liegt bei allen drei Modellen zwischen der deterministischen und der iterativen Methode.

Die Unterschiede zwischen den Methoden fallen damit beim Tokenverbrauch insbesondere bei der iterativen Methode deutlich aus. Den höchsten Wert der kombinierten Auswertung erreicht qwen3:8b mit der iterativen Methode mit durchschnittlich 12.149 Tokens.

6. Diskussion

- 3-4 Seiten
- wann welche Strategie sinnvoll ist
- Auswirkungen auf Energiegemeinschaften
- Limitationen

6.1. Brainstorming

- Die Ergebnisse deuten darauf hin, dass die Erstellung eines statischen Plans für das LLM einfacher ist als die iterative Steuerung eines Tool-gestützten Prozesses.
- Im plan-basierten Ansatz muss das Modell primär geeignete Tools auswählen und deren Verwendung planen.
- Im iterativen Ansatz muss das Modell zusätzlich den aktuellen Zustand des Prozesses berücksichtigen.
- `available_results` enthält formal die Informationen über bereits ausgeführte Tools, deren Argumente und deren Ergebnisse.
- Die explizite Bereitstellung dieser Informationen garantiert jedoch nicht, dass das LLM sie zuverlässig für die nächste Entscheidung nutzt.
- Tool Selection und State Tracking stellen damit unterschiedliche Anforderungen an das Modell dar.
- Ein Modell kann ein geeignetes Tool auswählen, ohne zuverlässig zu erkennen, dass dieses Tool bereits ausgeführt wurde.
- Die wiederholten Tool Calls des Qwen3:30B zeigen, dass das grundsätzliche Verständnis des iterativen Systems nicht automatisch zu einer effizienten Steuerung führt.
- Neben der Tool Selection stellt insbesondere die Fortschrittskontrolle eine Herausforderung dar.
- Das LLM muss erkennen, ob ein weiterer Tool Call tatsächlich neue Informationen liefert.
- Die Entscheidung, wann `generate_answer` aufgerufen werden soll, stellt eine eigene Herausforderung der iterativen Orchestrierung dar.
- Die bisherigen Beobachtungen deuten auf einen Einfluss der Modellgröße auf die Fähigkeit zur Steuerung iterativer Tool-Calling-Prozesse hin.
- Das größere Modell zeigt ein robusteres Verständnis des iterativen Zustands, löst jedoch Probleme wie redundante Tool Calls nicht vollständig.
- Modellgröße allein scheint daher nicht ausreichend zu sein, um eine robuste iterative Orchestrierung sicherzustellen.

- Deterministische Mechanismen des Orchestrators könnten bestimmte Fehlerklassen zuverlässiger verhindern als das LLM selbst.
- Dazu gehören beispielsweise die Erkennung identischer Tool Calls, Iterationslimits und Timeouts.
- Daraus ergibt sich die Möglichkeit eines hybriden Ansatzes, bei dem das LLM semantische Entscheidungen trifft und der Python-Orchestrator deterministische Kontrollaufgaben übernimmt.
- Die Beobachtungen sollten jedoch erst nach einer systematischen Evaluation als allgemeingültige Aussagen über Modellgrößen interpretiert werden.
- Die Ergebnisse von OrchDAG verdeutlichen zudem, warum eine getrennte Betrachtung verschiedener Bewertungsebenen sinnvoll ist. In den Experimenten erzielen Modelle teilweise eine deutlich höhere Accuracy/step als Accuracy/user_query. Die Autoren interpretieren dies dahingehend, dass Modelle einzelne Schritte korrekt ausführen können, gleichzeitig aber Schwierigkeiten haben, eine konsistente Gesamtstruktur der Tool-Ausführung aufrechtzuerhalten [18].

6.2. Warum die Tokenverbräuche unterschiedlich sind

- Deterministisch benötigt weniger Tokens, weil die Struktur einfacher ist und keine Formeln bzw. Planung erläutert werden müssen.
- Planbasiert benötigt etwas mehr Tokens durch die zusätzliche Planstruktur.
- Iterativ hat den höchsten Tokenverbrauch durch die höhere Komplexität, mehrere LLM-Aufrufe und den wachsenden Kontext.
- Die starke Streuung beim iterativen Ansatz hängt mit der unterschiedlichen Anzahl an Iterationen zusammen.
- Wichtig: Weniger Tokens sind nicht automatisch besser, da ein früher Abbruch bzw. fehlerhafter Durchlauf weniger Tokens verbrauchen kann als ein erfolgreicher Durchlauf mit mehreren Iterationen.

6.3. Bedeutung der hohen Fehlerrate bei der iterativen Methode

Beobachtung:

- Iterativ weist die höchste Fehlerrate auf.
- Iterativ benötigt mehr LLM-Aufrufe.
- Die einzelnen Iterationen werden teilweise nicht korrekt aufgelöst.
- Es treten redundante Toolaufrufe und erfundene result_ids auf.
- Dadurch kann der Prozess ohne Fortschritt weiterlaufen.

6.4. Zusammenhang zwischen Iterationen und Timeout

Hypothese: Bei Iterativ wird die Wahrscheinlichkeit für Timeout aufgrund höherer Aufrufe und steigender Komplexität erhöht.

- Iterative Methode hat höhere Fehlerrate.
- Iterative Methode hat höhere Laufzeit.
- Iterative Methode benötigt mehr Tokens.
- Mehrere LLM-Aufrufe und wachsende Kontexte könnten hierfür eine Erklärung darstellen.
- Ob tatsächlich die höhere Anzahl von Aufrufen die höhere Timeout-Wahrscheinlichkeit verursacht, ist eine Interpretation/Hypothese.

4. Probleme bei der Verarbeitung von Tool-Ergebnissen

Hierzu gehören insbesondere:

Schwierigkeiten treten vor allem nach der Verarbeitung erster Tool-Ergebnisse auf. Iterationen werden teilweise nicht korrekt aufgelöst. bereits ausgeführte Tools werden erneut aufgerufen. `available_results` verhindert redundante Tool Calls nicht zuverlässig. falsche Ergebnisse werden weitergegeben. statt weitere Tools aufzurufen, wird teilweise beschrieben, wie etwas berechnet werden könnte.

Das sind sehr interessante Fehlerbilder, die du in den Ergebnissen zwar knapp dokumentieren kannst. Die Interpretation, was diese Fehler über die jeweilige Orchestrierung aussagen, gehört aber in die Diskussion.

7. Fazit

- 2 Seiten

7.1. Brainstorming

- Lokale LLMs eignen sich grundsätzlich für Tool-gestützte Datenabfragen.
- Ein plan-basierter Ansatz ist für kleinere Modelle offenbar leichter umzusetzen als eine iterative Orchestrierung.
- Der iterative Ansatz stellt höhere Anforderungen an das LLM, da neben der Tool-Auswahl auch Zustandsverarbeitung, Fortschrittskontrolle und Terminierung erforderlich sind.
- Die explizite Bereitstellung bereits verfügbarer Ergebnisse reicht nicht zwangsläufig aus, damit kleinere LLMs diese zuverlässig in nachfolgenden Entscheidungen verwenden.
- Größere Modelle scheinen den iterativen Zustand besser erfassen zu können.
- Größere Modelle lösen jedoch nicht automatisch Probleme mit redundanten Tool Calls und zuverlässiger Terminierung.
- Eine Kombination aus LLM-basierter Entscheidungsfindung und deterministischer Kontrolle durch den Orchestrator erscheint daher als vielversprechender Ansatz.
- Laufzeit, Timeout und Iterationsanzahl müssen bei der Bewertung lokaler LLMs berücksichtigt werden.
- Eine faire Evaluation benötigt mehrere Kriterien, insbesondere Korrektheit, Effizienz und Robustheit.

7.2. Weiterführende Arbeiten

7.3. Brainstorming

- Untersuchung weiterer Modellgrößen.
- Vergleich weiterer lokaler LLMs.
- Untersuchung des Einflusses verschiedener Systemprompt-Formulierungen.
- Untersuchung expliziterer Zustandsrepräsentationen.
- Untersuchung einer deterministischen Duplikaterkennung für Tool Calls.
- Untersuchung unterschiedlicher Terminierungsstrategien.
- Vergleich eines vollständig LLM-gesteuerten Ansatzes mit einem hybriden Ansatz.
- Untersuchung des Einflusses des Iterationslimits.
- Untersuchung des Einflusses verschiedener Timeout-Werte.

- Untersuchung des Verhältnisses zwischen Modellgröße, Zuverlässigkeit und Laufzeit.
- Untersuchung des Einflusses der Aufgabenkomplexität auf die benötigte Modellgröße.
- Untersuchung, ob größere Modelle zwar zuverlässiger, aufgrund ihrer höheren Laufzeit aber weniger effizient sind.
- Zuverlässigkeit bei fehlenden Tools oder Nachfragen separat betrachten

Akronyme

Abbildungsverzeichnis

Abbildung 1 Vergleich der untersuchten Orchestrierungsstrategien	34
--	----

Tabellenverzeichnis

Tabelle 1	Technische Rahmenbedingungen	26
Tabelle 2	Für die Evaluation verwendete Energiedaten	27
Tabelle 3	Für die Evaluation bereitgestellte Werkzeuge	30
Tabelle 4	Modellauswahl	37
Tabelle 5	Bewertungskriterien	41
Tabelle 6	Gesamtvergleich der Methoden	47
Tabelle 7	Analyse der Korrektheits- und Effizienzkomponenten	48
Tabelle 8	Vergleich der Aufgabentypen	48
Tabelle 9	Vergleich der verwendeten Modelle	49
Tabelle 10	Vergleich der Modelle und Methoden	49

Source Code Verzeichnis

Literaturverzeichnis

- [1] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, und M. T. Ribeiro, „ART: Automatic Multi-Step Reasoning and Tool-Use for Large Language Models“. [Online]. Verfügbar unter: <https://arxiv.org/abs/2303.09014>
- [2] R. Yang u. a., „GPT4Tools: Teaching Large Language Model to Use Tools via Self-instruction“, in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, und S. Levine, Hrsg., Curran Associates, Inc., 2023, S. 71995–72007. [Online]. Verfügbar unter: https://proceedings.neurips.cc/paper_files/paper/2023/file/e393677793767624f2821cec8bdd02f1-Paper-Conference.pdf
- [3] Z. Duan und J. Wang, „Multi-Tool Integration Application for Math Reasoning Using Large Language Model“, in *2024 IEEE 10th International Conference on Edge Computing and Scalable Cloud (EdgeCom)*, 2024, S. 106–109. doi: 10.1109/EdgeCom62867.2024.00024.
- [4] L. Alazraki und M. Rei, „Meta-Reasoning Improves Tool Use in Large Language Models“, in *Findings of the Association for Computational Linguistics: NAACL 2025*, L. Chiruzzo, A. Ritter, und L. Wang, Hrsg., Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, S. 7900–7912. doi: 10.18653/v1/2025.findings-naacl.440.
- [5] Y. Huang u. a., „MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use“. [Online]. Verfügbar unter: <https://arxiv.org/abs/2310.03128>
- [6] J. Liu, K. Ning, Y. Su, W. Han, J. Xu, und Y. Zhang, „WTU-EVAL: A Whether-or-Not Tool Usage Evaluation Benchmark for Large Language Models“, in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, S. 1–5. doi: 10.1109/ICASSP49660.2025.10889153.
- [7] N. K. A. GmbH, „Was Sind Energiegemeinschaften Und Wie Funktionieren Sie?“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://www.next-kraftwerke.at/wissen/energiegemeinschaften>
- [8] Ö. K. für Energiegemeinschaften, „Rechtsgrundlagen Für Energiegemeinschaften“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: <https://energiegemeinschaften.gv.at/rechtsgrundlagen/>
- [9] Y. Zhang u. a., „Geospatial Large Language Model Trained with a Simulated Environment for Generating Tool-Use Chains Autonomously“, *International Journal of Applied Earth Observation and Geoinformation*, Bd. 136, S. 104312, 2025, doi: 10.1016/j.jag.2024.104312.
- [10] Z.-Y. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, und Y. Lin, „Towards Tool Use Alignment of Large Language Models“, in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, und Y.-N. Chen, Hrsg., Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, S. 1382–1400. doi: 10.18653/v1/2024.emnlp-main.82.

- [11] Akamai Technologies, „Was ist KI-Inferenz?“. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: <https://www.akamai.com/de/glossary/what-is-ai-inferencing>
- [12] H. Xu u. a., „The Evolution of Tool Use in LLM Agents: From Single-Tool Call to Multi-Tool Orchestration“. Zugegriffen: 10. April 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2603.22862>
- [13] G. Singh, P. Chhabra, und T. Banerjee, „A Comprehensive Review of LLM-based Text-to-SQL Systems: Methods, Datasets, and Trends“, *Computational Systems and Artificial Intelligence*, Bd. 2, Nr. 1, S. 1–6, Feb. 2026, doi: 10.69882/adba.csai.2026011.
- [14] Z. Hong u. a., „Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL“, *IEEE Transactions on Knowledge and Data Engineering*, Bd. 37, Nr. 12, S. 7328–7345, Dez. 2025, doi: 10.1109/TKDE.2025.3609486.
- [15] M. de Costa, M. Anwar, D. Mercier, M. Randall, und I. Hammad, „Enhancing Accuracy and Maintainability in Nuclear Plant Data Retrieval: A Function-Calling LLM Approach Over NL-to-SQL“. Zugegriffen: 15. August 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2506.08757>
- [16] H. Liu, C. Liu, und B. A. Prakash, „A Picture Is Worth A Thousand Numbers: Enabling LLMs Reason about Time Series via Visualization“.
- [17] T. Schick u. a., „Toolformer: Language Models Can Teach Themselves to Use Tools“. Zugegriffen: 24. März 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2302.04761>
- [18] Y. Lu, S. Liu, und L. Dong, „OrchDAG: Complex Tool Orchestration in Multi-Turn Interactions with Plan DAGs“. Zugegriffen: 16. April 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2510.24663>
- [19] T. Zhe u. a., „Robust and Efficient Tool Orchestration via Layered Execution Structures with Reflective Correction“. Zugegriffen: 26. März 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2602.18968>
- [20] A. Hernandez, V. Sabbia, und S. Perez, „ReAct Modular Agent: Orchestrating Tool-Use and Retrieval for Financial Workflows“, *Proceedings of International Conference on Intelligent Systems and New Applications*, S. 1, Dez. 2025, doi: 10.58190/icisna.2025.138.
- [21] B. Liu, G. Zhao, und H. Xu, „Utility-Guided Agent Orchestration for Efficient LLM Tool Use“. Zugegriffen: 9. April 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2603.19896>
- [22] S. G. Patil u. a., „The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models“, in *Forty-Second International Conference on Machine Learning*, 2025. [Online]. Verfügbar unter: <https://openreview.net/forum?id=2GmDdhBdDk>
- [23] „Handling Big Models for Inference c · Hugging Face“. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://huggingface.co/docs/accelerate/v0.32.0/concept_guides/big_model_inference
- [24] „Hardware/Software Environment for Performance Measurements — NVIDIA TensorRT“. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://docs.nvidia.com/deeplearning/tensorrt/10.x.x/performance/hw-sw-environment.html?utm_source=chatgpt.com
- [25] „Using Tools | OpenAI API“. Zugegriffen: 3. August 2026. [Online]. Verfügbar unter: <https://developers.openai.com/api/docs/guides/tools>

- [26] „GPT-5.4 Mini Model | OpenAI API“. Zugegriffen: 31. August 2026. [Online]. Verfügbar unter: <https://developers.openai.com/api/docs/models/gpt-5.4-mini>

